Data Structure

Liu Haiyang

Sorting Algorithm

Chapter 9 Contents

- Insertion Sort
- Exchange Sort (Bubble Sort, Quick Sort)
- Selection Sort (Simple Sort, Heap Sort)
- Merge Sort
- Radix Sorting
- A Comprehensive Comparison of Various Sorting Methods

Chapter 9

- **Key teaching points：**
  - **The characteristics of various sorting methods and their flexible application;**
  - **The sorting process of various methods;**
  - **Time complexity analysis of various sorting methods**
- **Difficult point in teaching：**
  - **The characteristics and time complexity analysis of various sorting methods**

Chapter Contents

- Insertion Sort
- Exchange Sort
- Selection Sort
- Merge Sort
- Radix Sorting
- A Comprehensive Comparison of Various Sorting Methods

Definition of sorting

- Sorting is a kind of operation which is often performed in computer science, and its purpose is to adjust a set of "**disorderly**" records to a "**ordered**" records.

For example: the following keyword sequence

52, 49, 80, 36, 14, 58, 61, 23, 97, 75

Adjust to

14, 23, 36, 49, 52, 58, 61 ,75, 80, 97

9.1 Overview

1. Definition of Sorting

2. Internal and External Sorting

3. Classification of Internal Sorting Methods

Generally speaking,

Assume the sequence with n records is $\{ R_1, R_2, \ldots, R_n \}$

The corresponding keyword sequence is $\{ K_1, K_2, \ldots, K_n \}$

These keywords can be compared with each other, that is, in

There is such a relationship between them:

$$\mathbf{K_{p1} \leqslant K_{p2} \leqslant \ldots \leqslant K_{pn}}$$

The operation of rearranging the record sequence into

$\{ R_{p1}, R_{p2}, \ldots, R_{pn} \}$

according to this inherent relationship is called **sorting**.

Stability of Sorting Algorithm

- If there are more than one record with the same keyword in the sequence to be sorted, the algorithm is **stable** if the relative order of the records remains unchanged after sorting, and is **unstable** if the relative order of the records changes after sorting.

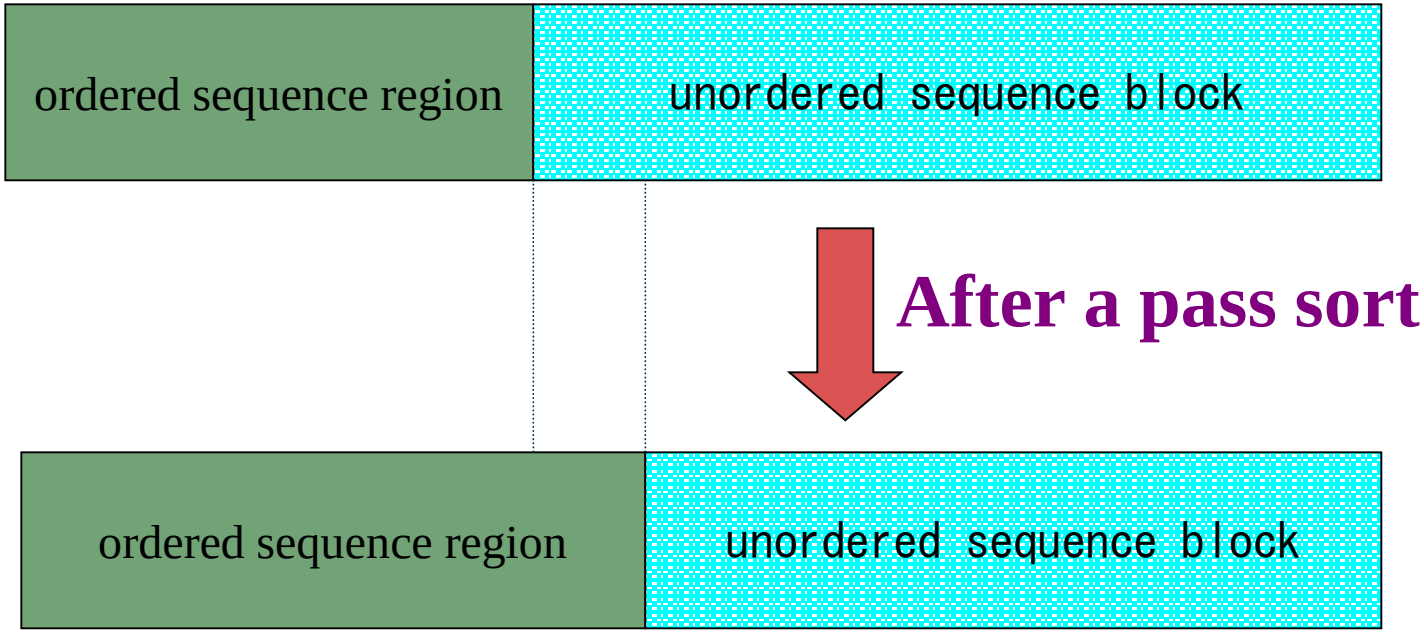
2. Internal and External Sorting

The process of sorting records stored in the computers random access memory (**RAM**) is called **internal sorting**.

If the number of records to be sorted is so large that the memory cannot hold all the records at once, the sorting process still needs to access the **external storage**, which is called **external sorting**.

3. Methods of Internal Sorting

The internal sort process is a process of **gradually expanding** the **length of the ordered sequence** of records.



- **Principles for sorting based on different criteria**
  - **insertion sort**
  - **exchange sort**
  - **selection sort**
  - **merge sort**
  - **radix sorting**
- **Workload required for sorting process**
  - **A simple sorting method with time complexity $O(n^2)$**
  - **An advanced sorting algorithm with O(nlogn) time complexity**
  - **Base sort has a time complexity of $O(d \cdot n)$.**

1. Insert category

Inserts one or more records from an unordered subsequence into an ordered subsequence to increase the length of the ordered subsequence.

2. Exchange class

By **exchanging** the records in the disordered sequence, the record with the minimum or maximum key is obtained and added to the ordered subsequence, thus increasing the length of the ordered subsequence.

3. Selected class

Select the record with the smallest or largest keyword from the unsorted subsequence and add it to the sorted subsequence to increase the length of the sorted subsequence.

4.Merger class

Merge two or more ordered record sub-sequences to gradually increase the length of the ordered record sequence.

5.Other methods

Definition of the data type for records to be sorted:

```
#define MAXSIZE  1000 //Maximum length of the sequential list to be sorted
typedef  int  KeyType;     // The key type is int

typedef  struct {
    KeyType   key;             // Key field
    InfoType  otherinfo;       // Other data fields
} RcdType;                     // Record type

typedef  struct {
    RcdType    r[MAXSIZE+1];   // r[0] is unused
    int        length;         // Length of the sequential list
} SqList;                      // Sequential list type
```

Chapter Contents

- **Insertion Sort**
- Exchange Sort
- Selection Sort
- Merge Sort
- Radix Sorting
- A Comprehensive Comparison of Various Sorting Methods

Basic Idea of One Pass of Direct Insertion Sort:

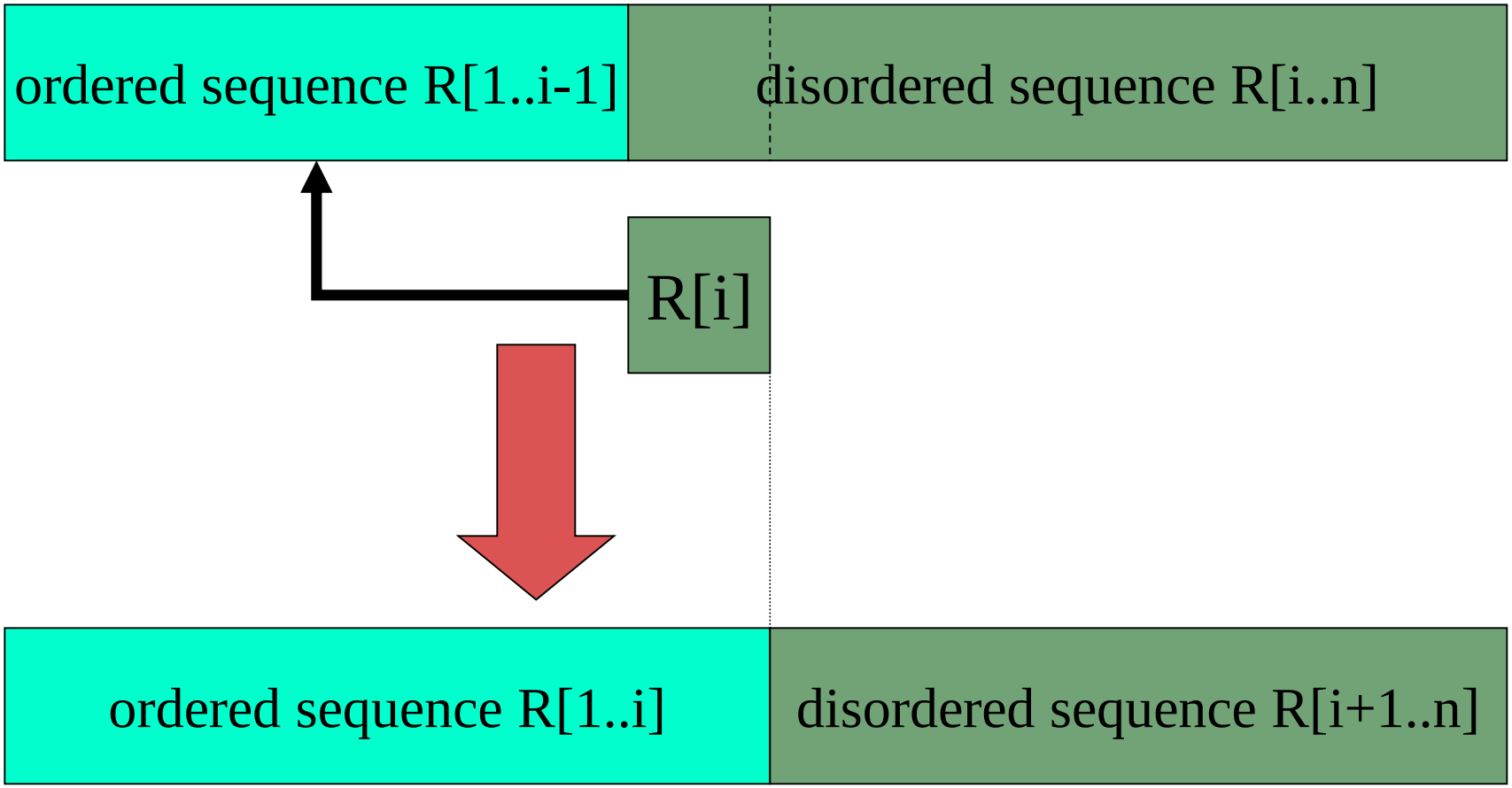
divide the sequence of n records into sorted and unsorted parts, that is, when it involves the i-th record R_i,

$$(R_1, \ldots, R_{i-1})$$

is the ordered portion that has been sorted

$$(R_i, R_{i+1}, \ldots, R_n)$$

This is an unsorted segment. Find the position where R i should be inserted in the ordered sequence, and insert R_i. Shift all records from the original position to R_i one position to the right.


ordered sequence R[1..i-1]
disordered sequence R[i..n]
R[i]
ordered sequence R[1..i]
disordered sequence R[i+1..n]

Implementing "one-pass insertion sort" can be done in three steps:

1. Find the insertion position of R[i] in R[1..i-1],

R[1..j].key ≤ R[i].key **<** R[j+1..i-1].key；

2. Replace R[j+1.. Move all records in [i-1]

backward one position;

3. Insert (copy) R[i] into the position of R[j+1].

Different implementation methods lead to different algorithm descriptions

- Direct Insertion Sort (Based on Sequential Search)
- Binary Insertion Sort (based on binary search)
- List Insertion Sort (based on linked list storage)
- Shell Sort (based on incrementally reducing the gap)

1. Direct Insertion Sort

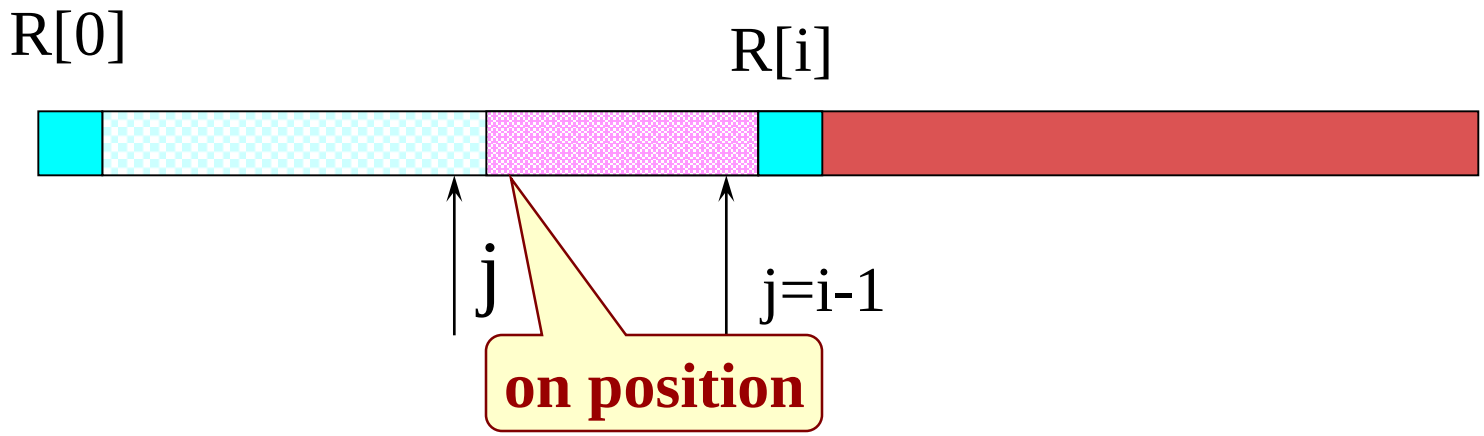
Implementation using **"sequential search"**

“Find the insertion position of R[i] in R[1..i-1]"

Key points for algorithm implementation:

- The sequential search is carried out from R[i-1] to R[0],

  and the sentinel is set at R[0].



R[0] = R[i]; ***// Set the sentinel***

for (j=i-1; R[0].key<R[j].key; --j);

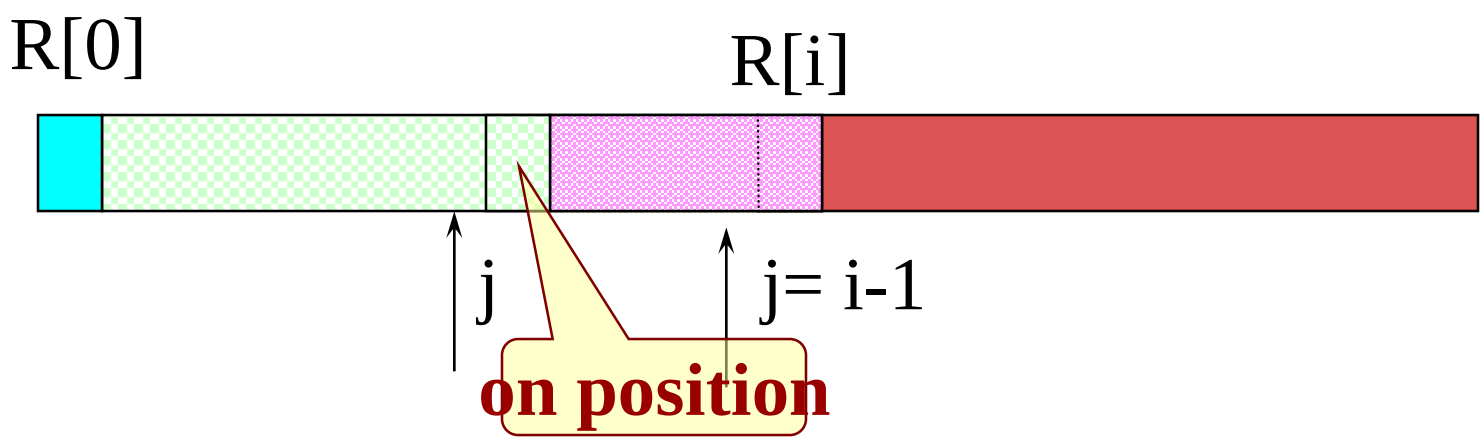
// Search from back to front

The loop ends, indicating that R[i] is inserted at position j + 1.

- for the records whose **keywords are not less than R[i].key** found in the search process, and **move the records backward** during the search;

for (j=i-1; R[0].key<R[j].key; --j);

R[j+1] = R[j]



You can insert directly after the loop ends

- set i to 2, 3,..., n,

  Sort the entire sequence.

```
for (  i=2;   i<=n;  ++ i )
if (R[i].key<R[i-1].key)
    {  Find the insertion position of R[i] in R[1..i-1];
       Insert R[i];
    }
```

Example of Direct Insertion Sort

he sequence to be sorted is (18, 12, 10, 12, 30, 16)Sequence state after each pass of simple insertion sort:

Initial state		[18] 12 10 <u>12</u> 30 16
Pass 1（i=2）	(12)	[12 18] 10 <u>12</u> 30 16
Pass 2（i=3）	(10)	[10 12 18] <u>12</u> 30 16
Pass 3（i=4）	(<u>12</u>)	[10 12 12 18] 30 16
Pass 4（i=5）	(30)	[10 12 12 18 30] 16
Pass 5（i=6）	(16)	[10 12 12 16 18 30]

Sentinel

```
void InsertionSort ( SqList &L ) {
                    // Insertion sort for the ordered list L
  for ( i=2; i<=L.length; ++i )
     if (L.r[i].key < L.r[i-1].key) {

        L.r[0] = L.r[i];              // Copy as sentinel
        for ( j=i-1; L.r[0].key < L.r[j].key;  -- j )
           L.r[j+1] = L.r[j];         // Shift record backward
        L.r[j+1] = L.r[0]; }  // Insert into the correct position

} // InsertSort
```

For direct insertion sort:

Best case scenario (Keywords are in ascending order in the record sequence):

Comparison count:

$$\sum_{i=2}^{n} 1 = n - 1$$

Number of "moves":

$$\mathbf{0}$$

worst case scenario (Keywords in descending order in the record sequence):

Comparison count:

$$\sum_{i=2}^{n} i = \frac{(n+2)(n-1)}{2}$$

Number of "moves":

$$\sum_{i=2}^{n} (i+1) = \frac{(n+4)(n-1)}{2}$$

Generally, the records to be sorted are random, that is, the probability of various permutations of the records in the column to be sorted is the same, then the average of the minimum value and the maximum value can be taken as the number of times to compare the key and the number of times to move the records in direct insertion sort, which is about n 2/4.

For direct insertion sort: its **time complexity is $O(n^2)$**

Applicable when **the number of records to sort is small**

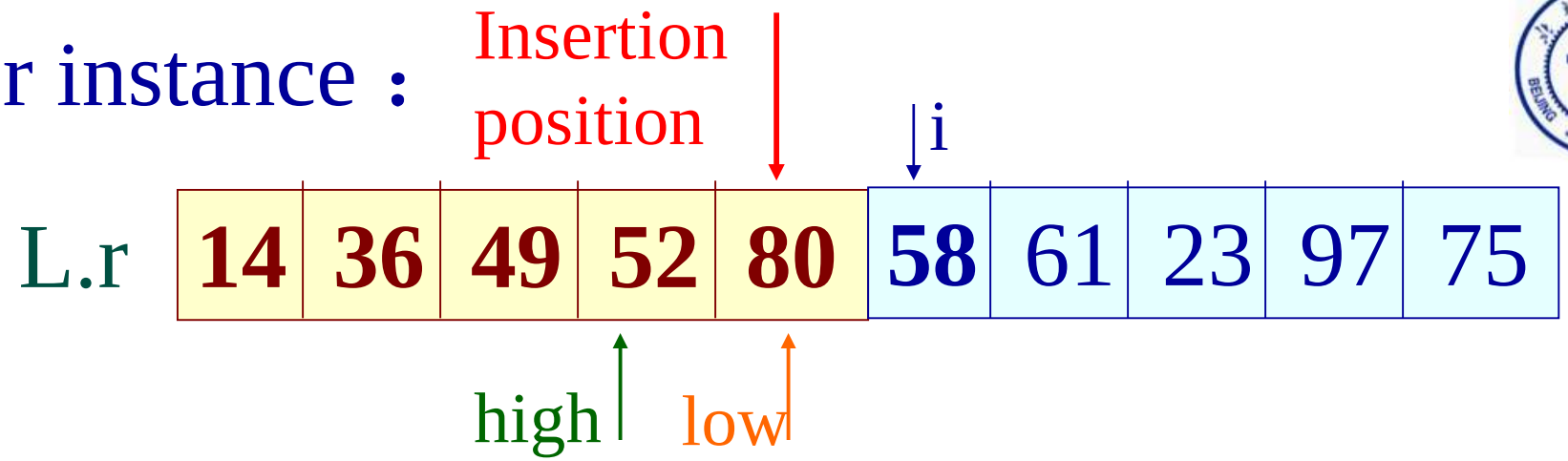
2. Binary Insertion Sort

Since R[1..i-1] is a sorted sequence ordered by key, we can use **binary search** to **find** the **insertion position** of R[i] in R[1..i-1]. The insertion sort implemented in this way is called **binary insertion sort**.

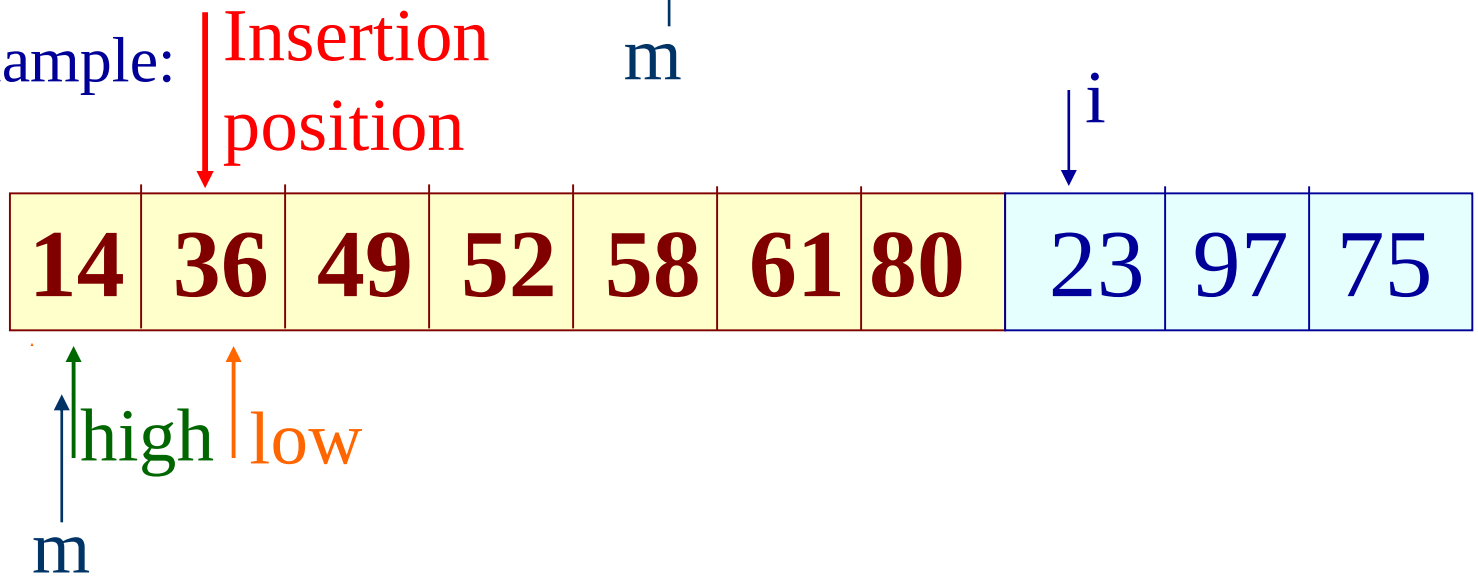
```
void BiInsertionSort ( SqList &L ) {
  // performs a binary insertion sort on the sorted list L
  for ( i=2; i<=L.length; ++i ) {
      L.r[0] = L.r[i];    // Temporarily store L.r[i] in L.r[0]
Binary search for the insertion position (high+1) in L.r[1..i-1]
      for ( j=i-1;  j>=high+1;  --j )
         L.r[j+1] = L.r[j];        // Shift records backward
      L.r[high+1] = L.r[0];    // Insert
  } // for

} // BInsertSort
```

```
// Binary search for the insertion position (high+1) in L.r[1..i-1]

low = 1;   high = i-1;
while (low<=high) {
    m = (low+high)/2;        // Split the interval in half
    if (L.r[0].key < L.r[m].key)
        high = m-1;   // Insertion point is in the lower half
    else  low = m+1;   // Insertion point is in the upper half
}
```

for instance：



Another example:



The time complexity of half-insertion sort is $O(n^2)$.

The split insertion sort is suitable for the large number of records to be sorted, and can greatly reduce the number of key words comparison.

3. List Insertion Sort

To reduce the "shifting" of records during the sorting process, we need to change the storage structure used in sorting.
We perform sorting using a **static linked list**, and after sorting is complete, we adjust the positions of all records in one batch—moving each record to its correct position.

	0	1	2	3	4	5	6	7	8
initial status	MAXINT	49	38	65	97	76	13	27	52
	1	0	-	-	-	-	-	-	-

	MAXINT	49	38	65	97	76	13	27	52
i=2	2	0	1	-	-	-	-	-	-

Key field

Next field

	MAXINT	49	38	65	97	76	13	27	52
i=3	2	3	1	0	-	-	-	-	-

	MAXINT	49	38	65	97	76	13	27	52
i=4	2	3	1	4	0	-	-	-	-

	0	1	2	3	4	5	6	7	8
i=5	MAXINT	49	38	65	97	76	13	27	52
	2	3	1	5	0	4	-	-	-

i=6	MAXINT	49	38	65	97	76	13	27	52
	6	3	1	5	0	4	2	-	-

i=7	MAXINT	49	38	65	97	76	13	27	52
	6	3	1	5	0	4	7	2	-

i=8	MAXINT	49	38	65	97	76	13	27	52
	6	8	1	5	0	4	7	2	3

```
void LInsertionSort (Elem SL[ ],   int n){
    // Insertion sort for record sequence SL[1..n]
  SL[0].key = MAXINT ;
//The 0th component is the header node
  SL[0].next = 1;  SL[1].next = 0;
  for ( i=2; i<=n; ++i )
     for ( j=0, k = SL[0].next;SL[k].key<
              SL[i].key ; j=k, k=SL[k].next );
     SL[j].next = i;  SL[i].next = k;
              //Insert node i between nodes j and k
}// LinsertionSort
```

The result of list insertion sort is an ordered linked list, which only supports sequential search (not random access). To enable binary search on the ordered list, we must reorder the records.

How to adjust the record sequence after sorting?

The algorithm uses three pointers:

p: Points to the current position of the i-th record

i: Points to the target position of the i-th record

q: Points to the current position of the (i+1)-th record

	0	1	2	3	4	5	6	7	8
initial status	MAXINT	49	38	65	97	76	13	27	52
	6	8	1	5	0	4	7	2	3

i=1 p=6 q=7	MAXINT	13	38	65	97	76	49	27	52
	6	6	1	5	0	4	8	2	3

Key field

Next field

i=2 p=7 q=2	MAXINT	13	27	65	97	76	49	38	52
	6	6	7	5	0	4	8	1	3

i=3 p=(2) p=7 q=1	MAXINT	13	27	38	97	76	49	65	52
	6	6	7	7	0	4	8	5	3

	0	1	2	3	4	5	6	7	8
i=4 p=(1) p=6 q=8	MAXINT	13	27	38	49	76	97	65	52
	6	6	7	7	6	4	0	5	3

i=5 p=8 q=3	MAXINT	13	27	38	49	52	97	65	76
	6	6	7	7	6	8	0	5	4

i=6 p=(3) p=7 q=5	MAXINT	13	27	38	49	52	65	97	76
	6	6	7	7	6	8	7	0	4

i=7 p=(5) p=8 q=4	MAXINT	13	27	38	49	52	65	76	97
	6	6	7	7	6	8	7	8	0

```
void Arrange ( Elem SL[ ], int n ) {
  p = SL[0].next;
                    // p indicates the current position of the first record
  for ( i=1; i<n; ++i ) {
    while (p<i)  p = SL[p].next;
                                //All elements less than i have been sorted
    q = SL[p].next;
          // q points to the end of the table that hasnt been adjusted yet
   if ( p!= i ) {
      SL[p]←→SL[i];
                    //Swap records to move the i-th record to the front
      SL[i].next = p;     // Pointer to the removed record
    }
    p = q;                  // p indicates the unadjusted end of the table,
                            // Prepare to find the i+1th record
  }
} // Arrange
```

4. Shell Sort (Also Known as Diminishing Increment Sort)

Basic Idea: First perform a "macro" adjustment on the sequence of records to be sorted, then a "micro" adjustment.

The so-called "macro" adjustment refers to jump-style insertion sort: split the entire sequence into several sub-sequences, perform direct insertion sort on each sub-sequence separately. When the entire sequence becomes basically ordered, perform a final direct insertion sort on all records.

The specific method is to divide the record sequence into several sub-sequences and then perform insertion sort on each sub-sequence.

For example: Divide n records into d sub-sequences:

{ R[1], R[1+d], R[1+2d], …, R[1+kd] }

{ R[2], R[2+d], R[2+2d], …, R[2+kd] }

…

{ R[d], R[2d], R[3d], …, R[kd], R[(k+1)d] }

Here, d is the increment, whose value decreases progressively from largest to smallest during the sorting process until it is reduced to 1 in the final pass.

Initial keywords 49 38 65 97 76 13 27 49 55 04

Set increment **d =5**

49 13

38 27

65 49

97 55

76 04

A sorted result: 13 27 49 55 04 49 38 65 97 76

Set increment **d =3**

13 55 38 76

27 04 65

49 49 97

Second sort result 13 04 49 38 27 49 55 65 97 76

Set increment **d =1**

Third sort result 04 13 27 38 49 49 55 65 76 97

```
void ShellInsert ( SqList &L, int dk ) {
        // Perform a heap sort with increment dk on the sorted list L
  for ( i=dk+1; i<=n; ++i )
    if ( L.r[i].key< L.r[i-dk].key)
   { L.r[0] = L.r[i];
// Temporarily store the current element in R[0]
      for (j=i-dk;  j>0&&(L.r[0].key<L.r[j].key); j-=dk)
          L.r[j+dk] = L.r[j];
// Shift elements backward to find the insertion position
      L.r[j+dk] = L.r[0];     // Insert the element
    } // if
} // ShellInsert
```

```
void ShellSort (SqList &L, int dlta[], int t)
{    // Perform Shell Sort on the sequential list L.,
  // dlta[0..t-1] stores the increment sequence (e.g., 5, 3, 2, 1)
    for (k=0; k<t; ++k)
        ShellInsert(L, dlta[k]);
 //Perform one pass of insertion sort with increment dlta[k]
} // ShellSort
```

Analysis of Shell Sort

- **From the above sorting process, it can be seen that the sub-sequences in Shell Sort are not formed by simply "segmenting sequentially". Instead, a sub-sequence consists of records separated by a certain "increment".Records with smaller keys are moved forward in a leapfrog manner (rather than step by step). This means that when the final pass (with increment = 1, which is a standard insertion sort) is performed, the sequence is already nearly ordered. Only a small number of comparisons and moves are needed to complete the sorting.Thus, the time complexity of Shell Sort is lower than that of direct insertion sort. Its time efficiency is a function of the chosen increment sequence.**

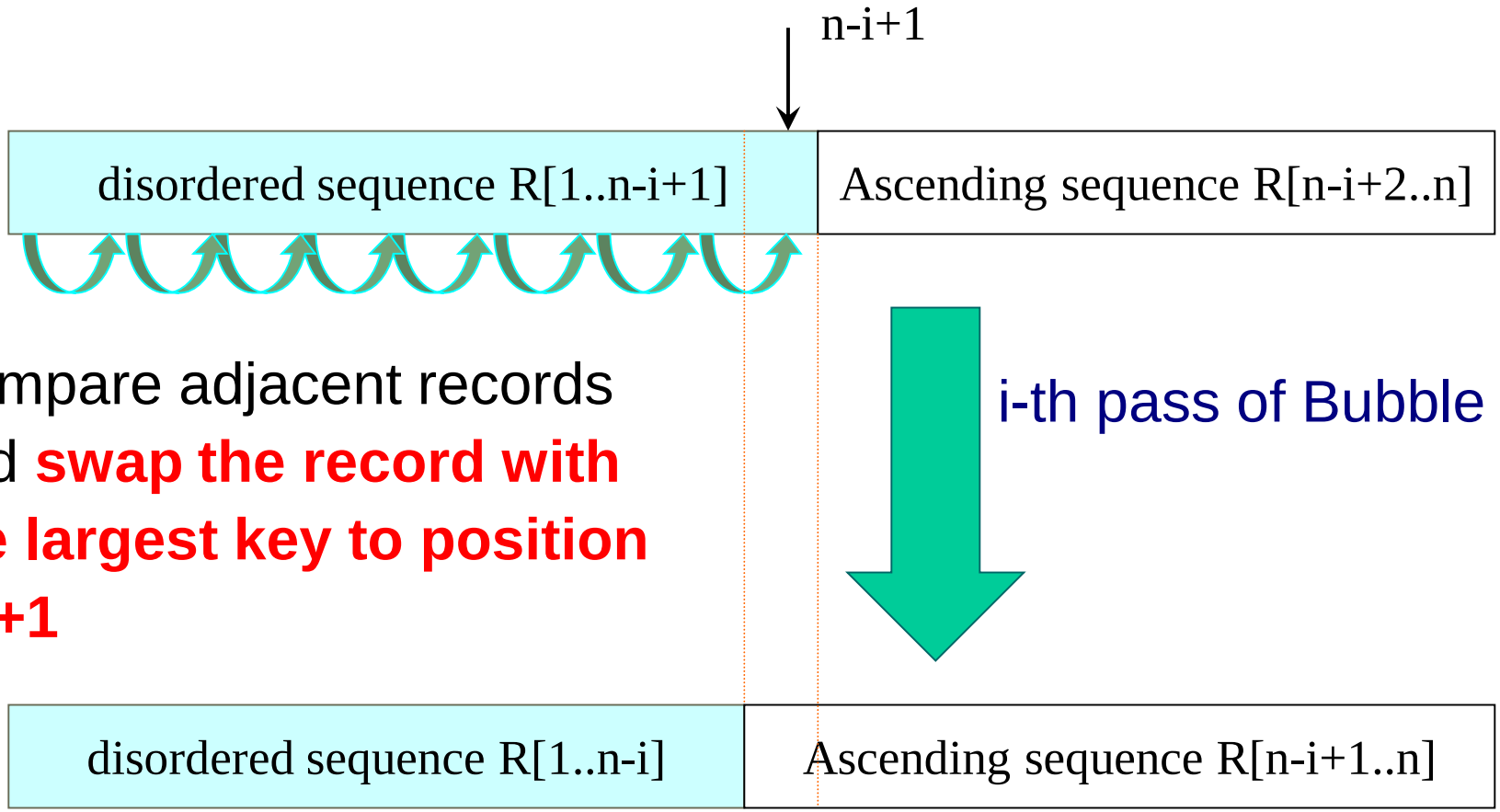
Chapter Contents

- **Insertion Sort**
- **Exchange Sort**
  - **Bubble Sort**
  - **One-Pass Quick Sort**
  - **Quick Sort and Its Analysis**
- Selection Sort
- Merge Sort
- Radix Sorting
- A Comprehensive Comparison of Various Sorting Methods
- External Sort

Bubble Sort

- Starting from R[1], **pairwise compare** the keys of R[i] and R[i+1]. If R[i].key> R[i+1].key, swap R[i] and R[i+1].key. After the first round of comparisons, R[n] is the **largest** record in the sequence.
- Then, starting from R[1], compare R[i] with R[i+1] (i=1,2,...,n-2) in pairs. If R[i].key> R[i+1].key, swap R[i] and R[i+1].key. After the second round of comparisons, R[n-1] becomes the **second-largest** record in the sequence.
- **After n-1 iterations of bubble sort,** the n records to be sorted will be in order of their keys.

Assume that during the sorting process, the status of the record sequence R[1..n] is:



Compare adjacent records and **swap the record with the largest key to position n-i+1**

Bubble Sort Example

Set the key sequence of the pending records as (65,97,76,13,27,49,58)
The bubble sort sequence state after each execution is as follows:

Initial state	[65 97 76 13 27 49 58]
1st pass (j=1~6)	[65 76 13 27 49 58] **97**
2nd pass (j=1~5)	[65 13 27 49 58] **76 97**
3rd pass (j=1~4)	[13 27 49 58] **65 76 97**
4th pass (j=1~3)	[13 27 49] **58 65 76 97**
5th pass (j=1~2)	[13 27] **49 58 65 76 97**
6th pass (j=1)	[13] **27 49 58 65 76 97**

Bubble Sort Algorithm

```
void bubblesort(Elem R[ ], int n)
    // Bubble sort for sequential list R[1.. n]
  {
      for(i=1;i<n;i++)
        for(j=1;j<=n-i;j++)
          if(R[j].key>R[j+1].key)
            {Swap(R[j], R[j+1]);} //Swap elements
}
```

Improvement of Bubble Sort

The algorithm described above requires n-1 iterations of bubble sort for a sequence of n records. As shown in the example, the sequence becomes sorted after the third iteration, rendering the subsequent three iterations redundant (no swaps occur). To optimize the algorithm, we should implement a swap-tracking mechanism: if no swaps are detected during any iteration, the sequence is considered sorted, and the sorting process should terminate.

Improved Bubble Sort Algorithm

```
void bubblesort(Elem R[ ], int n)
                    // Improved Bubble Sort Algorithm 1
 { i=n;   flag=1;// flag=1 indicates an exchange has occurred
   while(( i>1) && flag ))
     { flag=0;
       for(j=1;j<i;j++)
         if(R[j].key>R[j+1].key)
             {Swap(R[j], R[j+1]); flag=1; }
       i --;
     }
}
```

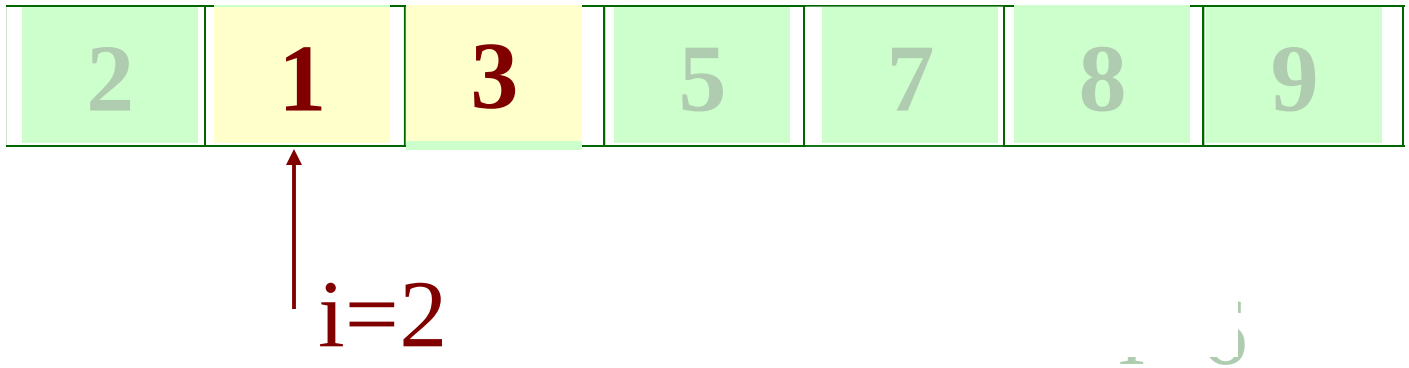
The bubble sort terminates when
The last transaction did not perform an "exchange record"

Further Improvement of Bubble Sort

Typically, each bubble cycle triggers i minus 1**, though this pattern isnt always observed.**

Further improvement: By remembering the position of the last swapped record in the current pass, the number of comparisons can be reduced in the next bubble pass.

for instance :



Time analysis:

Best case scenario (Keywords are in order in the record sequence):

Just one bubble is needed

Comparison count:

n-1

Number of moves:

0

Worst-case scenario (keys in reverse order):

n-1 bubbles needed

Comparison count:

$$\sum_{i=n}^{2}(i-1)=\frac{n(n-1)}{2}$$

Number of moves:

$$3\sum_{i=n}^{2}(i-1)=\frac{3n(n-1)}{2}$$

Chapter Contents

- **Insertion Sort**
- **Exchange Sort**
  - **Bubble Sort**
  - **One-Pass Quick Sort**
  - **Quick Sort and Its Analysis**
- Selection Sort
- Merge Sort
- Radix Sorting
- A Comprehensive Comparison of Various Sorting Methods
- External Sort

2. One-pass quick sort (one partition)

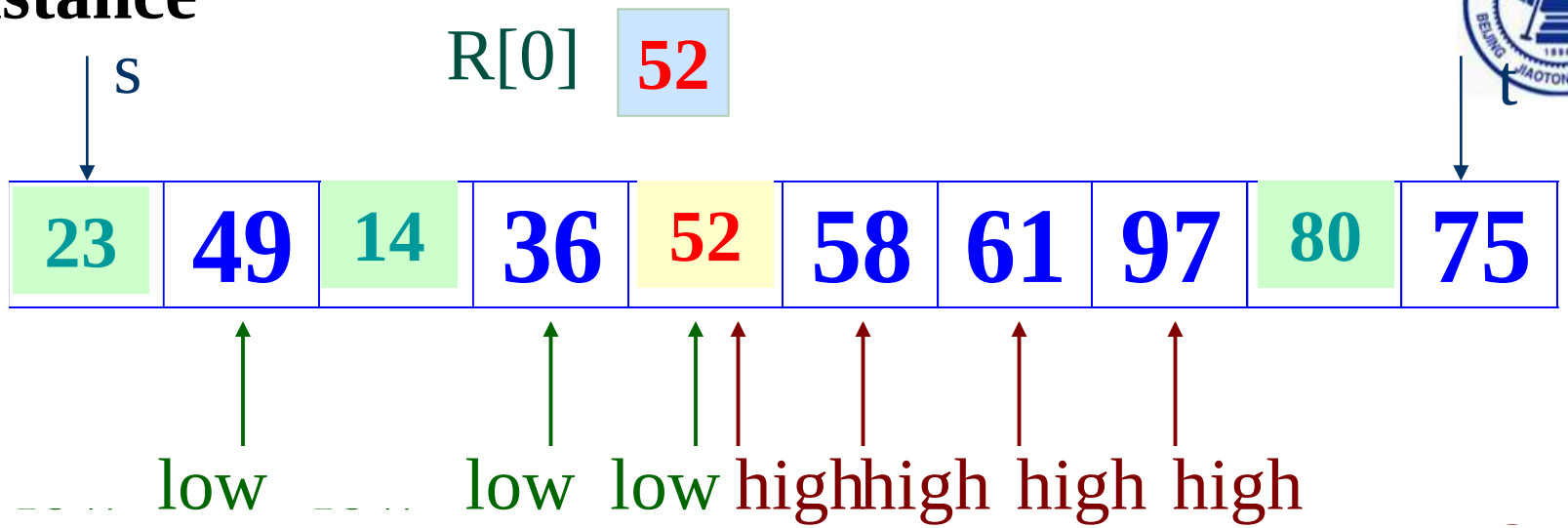
Objective: Identify a record and use its keyword as the **pivot**. All records with **keywords smaller than the pivot** are **moved before this record**, while **those with keywords larger are moved after it.**

As a result, **after a sort**, the unordered sequence R[s..t] will be **divided into two parts**: R[s..i-1] and R[i+1..t], and

R[j].key≤ R[i].key ≤ R[j].key

(s≤j≤i-1) pivot (i+1≤j≤t).

for instance



Let R[s]=52 be the pivot

Compare R[high].key with the pivot key, requiring R[high].key ≥ the pivot key

Compare R[low].key with the pivot key, requiring R[low].key ≥the pivot key

As shown, after the **one partition**, the keyword sequence

52, 49, 80, 36, 14, 58, 61, 97, 23, 75

Adjust to: 2 **23**, 49, **14**, 36, **(52)** 58, 61, 97, **80**, 75

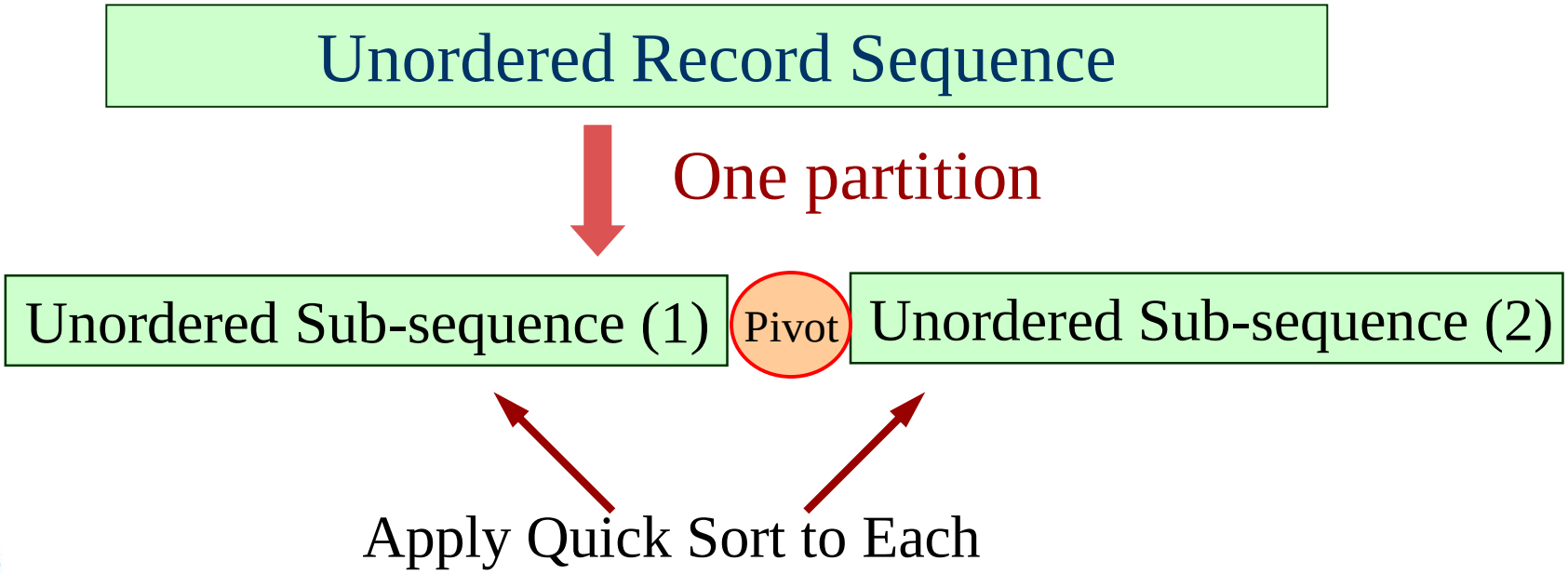
During the adjustment process, two pointers—**low** and **high**—were created, with initial values s and t respectively. First, gradually decrease the **high** value, then increase the low value, while ensuring that R[**high**].key ≥ 52 and R[**low**].key ≤ 52. If not, perform a record swap. Repeat the above steps...

```
int Partition (RedType& R[], int low, int high) {
                    //  Do a quick sort of R[low,..high]
  pivotkey = R[low].key;
  while (low<high) {
    while (low<high && R[high].key>=pivotkey)
      --high;
    R[low] ↔ R[high];
    while (low<high && R[low].key<=pivotkey)
      ++low;
    R[low] ↔ R[high];
  }
   return low;          // Return the pivot position
} // Partition
```

```
int Partition (RedType R[], int low, int high) {
    //Improvement of a Quick Sort Algorithm
R[0] = R[low];  pivotkey = R[low].key;  // Pivot element
while (low<high) {
   while(low<high&& R[high].key>=pivotkey) -- high;
     R[low] = R[high];          // Search from right to left
   while (low<high && R[low].key<=pivotkey) ++ low;
     R[high] = R[low]; }    // Search from left to right
R[low] = R[0];     return low;
}// Partition
```

3. Quick Sort

First, perform a "**one partition**" on the unordered record sequence. Then, **recursively apply quick sort** to the two sub-sequences obtained from the partition.



```
void QSort (RedType & R[], int s, int t ) {
 // Sort the record sequence R[s..t] quickly
   if (s < t) {                    // Length is greater than 1
     pivotloc = Partition(R, s, t);
                                   // Divide R[s..t] once
     QSort(R, s, pivotloc-1);
   // Recursively sort the lower subsequence,
   // pivotloc is the pivot position
     QSort(R, pivotloc+1, t); // Recursively sorts the high subsequence
 }
} // QSort
```

When the function Qsort is called for the first time, the upper and lower bounds of the sequence to be sorted are 1 and L.length, respectively.

```
void QuickSort( SqList & L) {
                                    //Sort a sequential list quickly
    QSort(L.r, 1, L.length);
} // QuickSort
```

4. Time Analysis of Quick Sort

Assuming the pivot position i=k obtained from a single partition, the time required for quicksort on n records is:

$$T(n) = T_{pass}(n) + T(k-1) + T(n-k)$$

$T_{pass}(n)$ denotes the time required to partition n records.

If the records in the sorted column have randomly distributed keywords, ***the probability of k being any value from 1 to n is equal.***

Thus, the average time required for quicksort is:

$$T_{avg}(n) = Cn + \frac{1}{n}\sum_{k=1}^{n}\left[T_{avg}(k-1) + T_{avg}(n-k)\right]$$

Let T avg (1) ≤ b

The result is obtained:

$$T_{avg}(n) < (\frac{b}{2} + 2c)(n+1)\ln(n+1)$$

Conclusion: The time complexity of quick sort is O(nlogn).

If the initial state of the queue is ordered by key, the quick sort will degenerate into bubble sort, and its time complexity is $O(n^2)$.

To prevent this from happening, perform a "pre-process" before any division:

First, compare R(s).key, R(t).key and $R[\lfloor (s+t)/2 \rfloor]$.key, and then **take** the key as the record of "the three" is the **pivot record.**

Chapter Contents

- **Insertion Sort**
- **Exchange Sort**
- **Selection Sort**
  - **Bubble Sort**
  - **One-Pass Quick Sort**
  - **Quick Sort and Its Analysis**
- Merge Sort
- Radix Sorting
- A Comprehensive Comparison of Various Sorting Methods
- External Sort

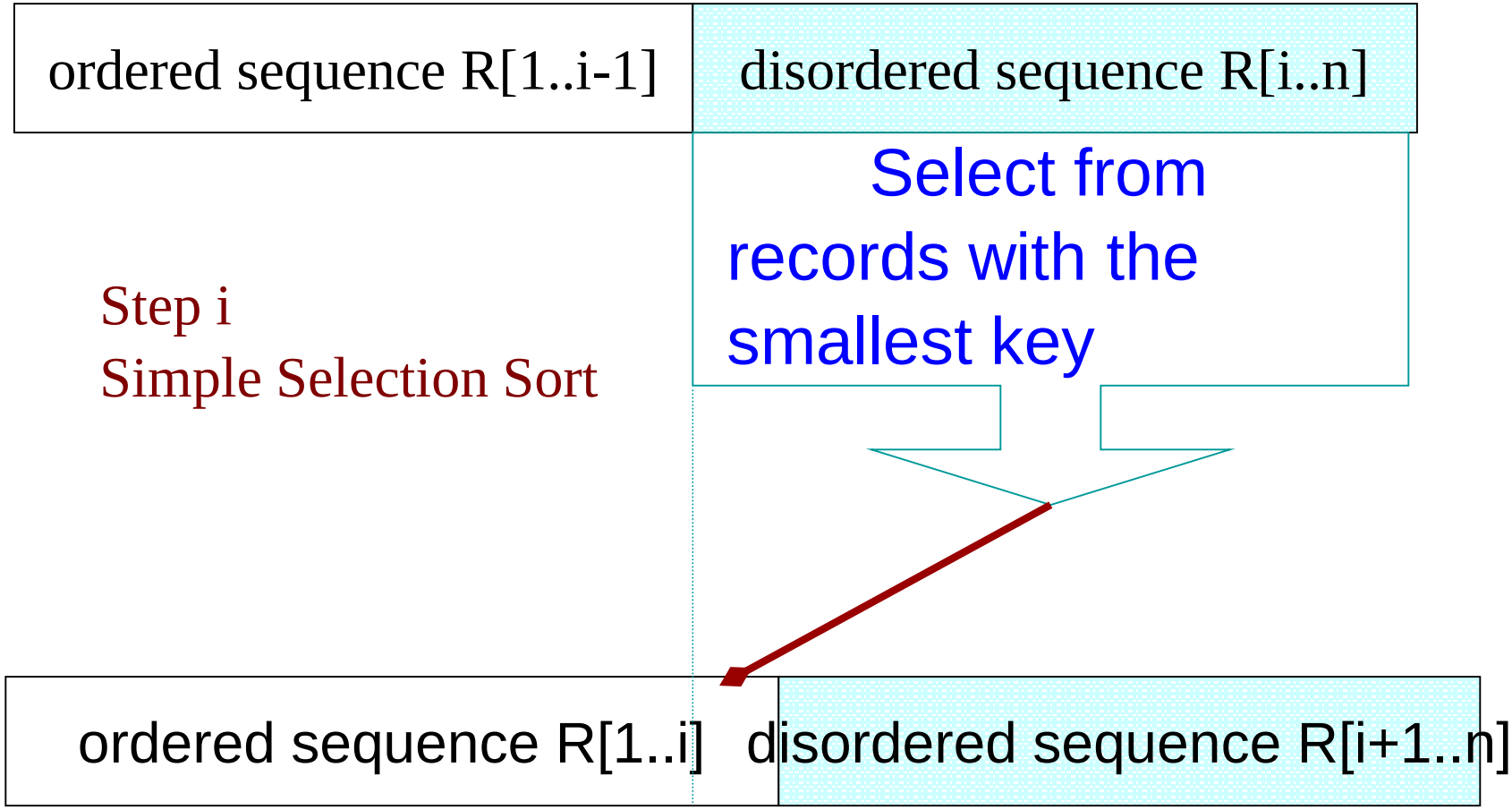
1. Simple Selection Sort

Basic idea:

A simple selection sort operation involves comparing n-i key values, selecting the record with the smallest key from n-i+1 records, and swapping it with the i-th record (1≤i≤n).

The algorithm for simple selection sort of records in L.r[1..n] is as follows: Let i be from 1 to n-1, and perform n-1 selection operations.

Assume the status of the sorted record sequence during sorting is:



Simple Selection Sort

The key sequence of the records to be sorted is (2, 7, 2, 4, 3, 1)
Simply select the sequence state after each execution:

Initial state	[2	7	2	4	3	1]	
Pass 1（i=1）	1	[7	2	4	3	2]	(j=6)
Pass 2（i=2）	1	2	[7	4	3	2]	(j=3)
Pass 3（i=3）	1	2	2	[4	3	7]	(j=6)
Pass 4（i=4）	1	2	2	3	[4	7]	(j=5)
Pass 5（i=5）	1	2	2	3	4	[7]	(j=5)

The simple selection sort algorithm is described as follows:

```
void SelectSort (Elem R[], int n ) {
  // Do a simple selection sort of the record sequence R[1..n]
  for (i=1; i<n; ++i) {
    // Select the i-th smallest record and swap it
    j = SelectMinKey(R, i);
    // Select the record with the smallest keyword in R[i..n]
    if (i!=j)  R[i]←→R[j];
                    // Swap with the i-th record
  }
} // SelectSort
```

Time Performance Analysis

For simple selection sort on n records, the total number of **key comparisons required** is:

$$\sum_{i=1}^{n-1}(n-i)=\frac{n(n-1)}{2}$$

Move the record count, with a minimum of 0 and a maximum of 3(n-1)**.**

Its time complexity is O(n²).

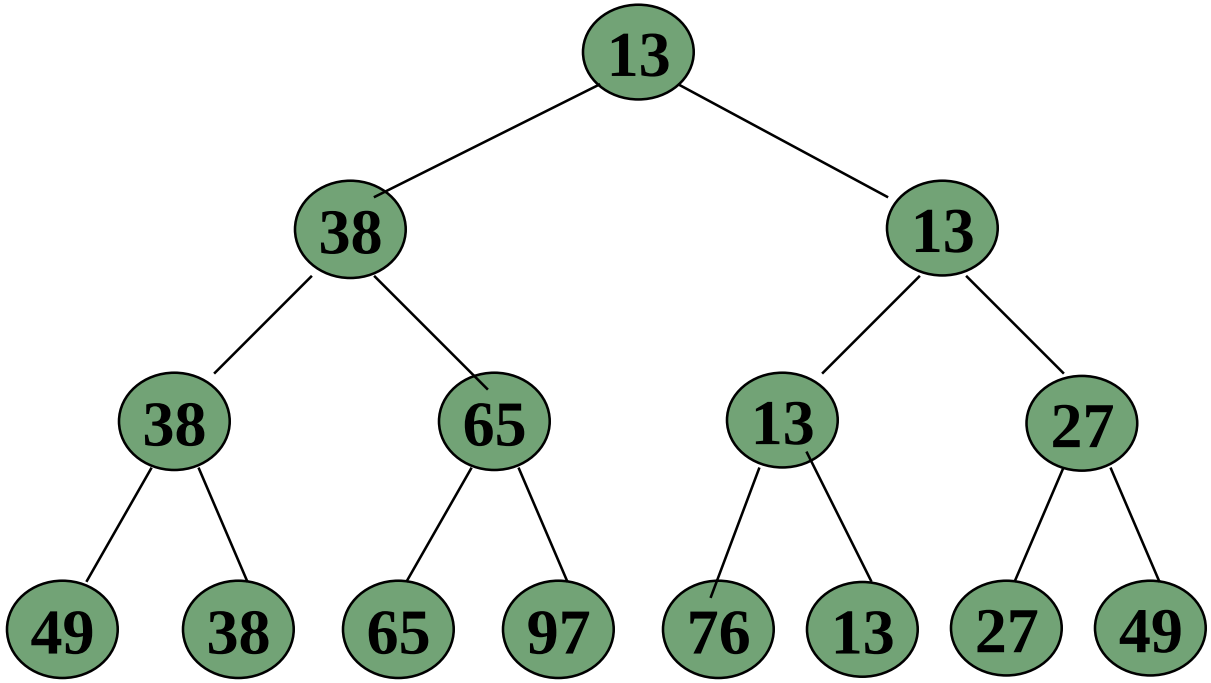
2. Tree Selection Sort

Tree Selection Sort (also called Tournament Sort) is a method of selection sort based on the idea of tournament.

The basic idea is to compare the keywords of n records in pairs, and then compare the n/2 smaller ones in pairs, and so on until the record with the smallest keyword is selected.

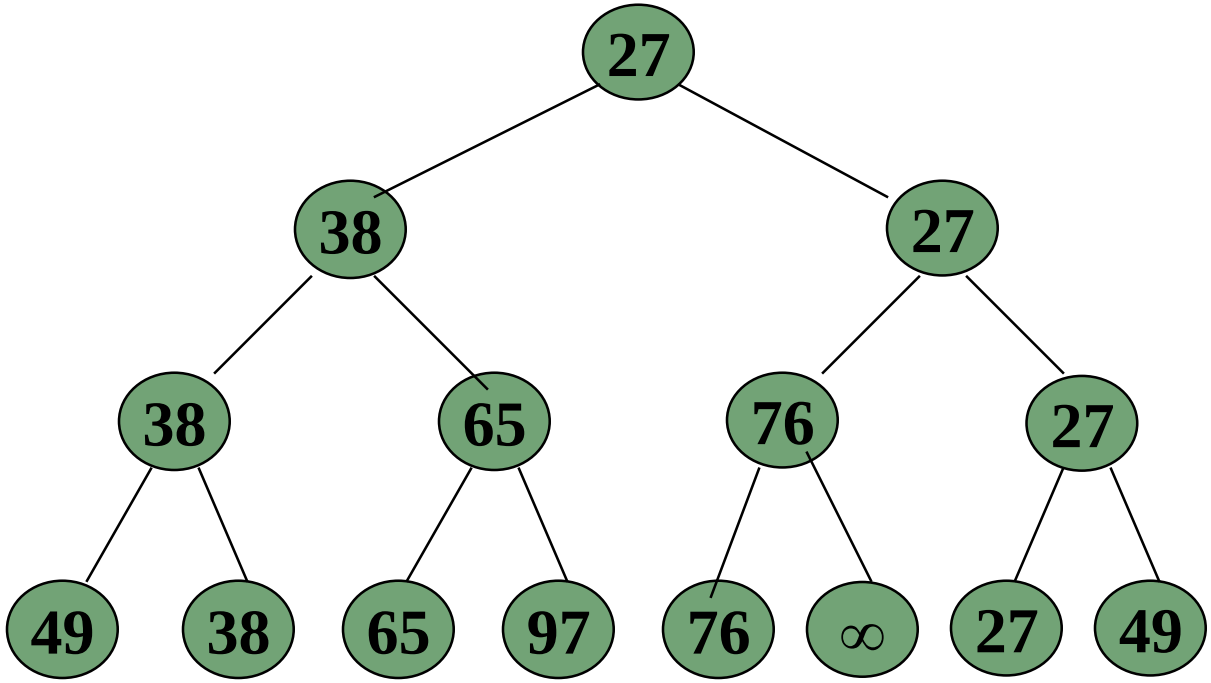
Example： 49 38 65 97 76 13 27 49

Select the smallest key 13



13

Replace the minimum key (13) in the leaf node with the maximum value, then update the keys of all nodes along the path from the leaf node to the root. The key of the root node will then be the second smallest key. Thus, the second smallest key is selected as 27.

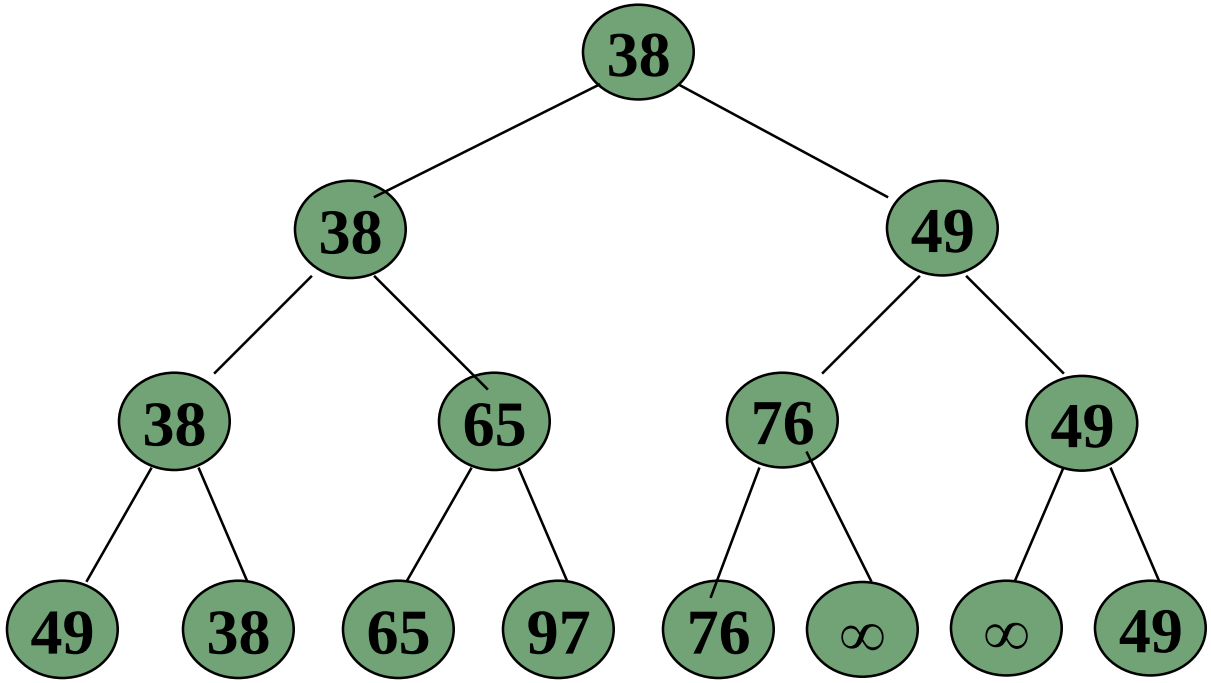


13

27

Similarly, you can select all keywords in ascending order. The third keyword is 38.

……



13

27

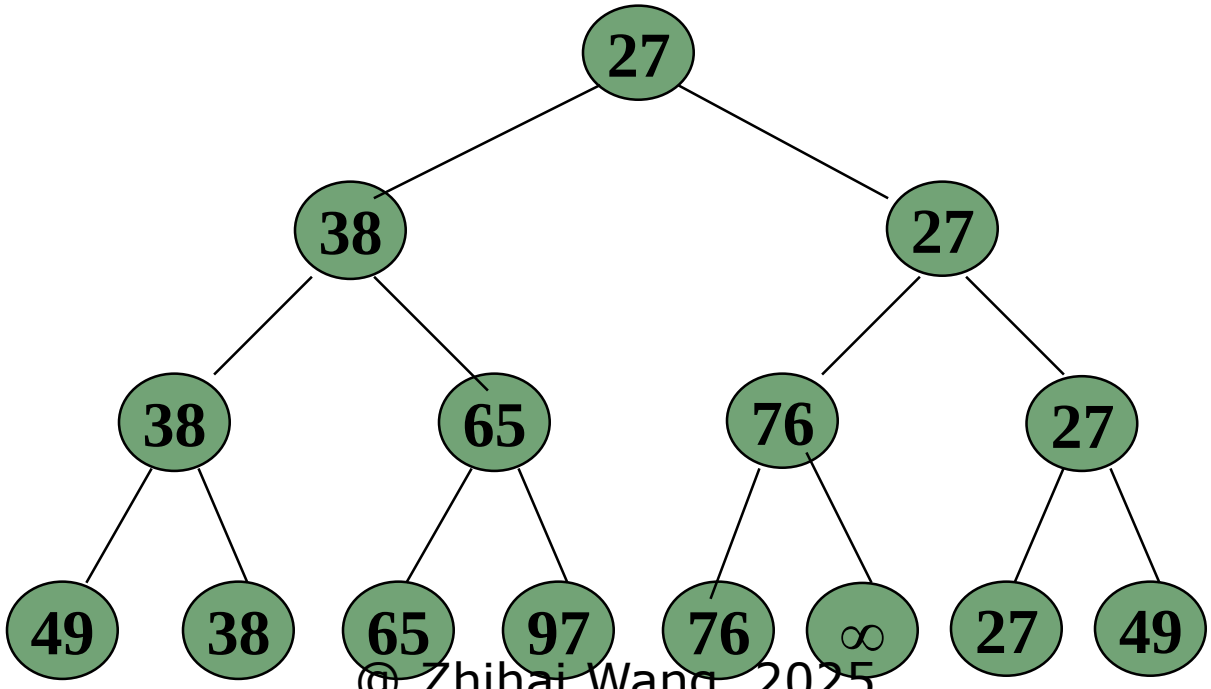
38

Time Complexity Analysis of Tree Selection Sort:

For a complete binary tree with n leaf nodes, its depth is $\lceil \log_2 n \rceil + 1$.

In tree selection sort, except for selecting the smallest key, each subsequent selection of the next smallest key only requires $\lceil \log_2 n \rceil$ comparisons.

Therefore, the time complexity of tree selection sort is $O(n\log_2 n)$.



3. Heap Sort

Heap definition:

A heap is a sequence $\{r_1, r_2, \ldots, r_n\}$ that satisfies one of the following properties:

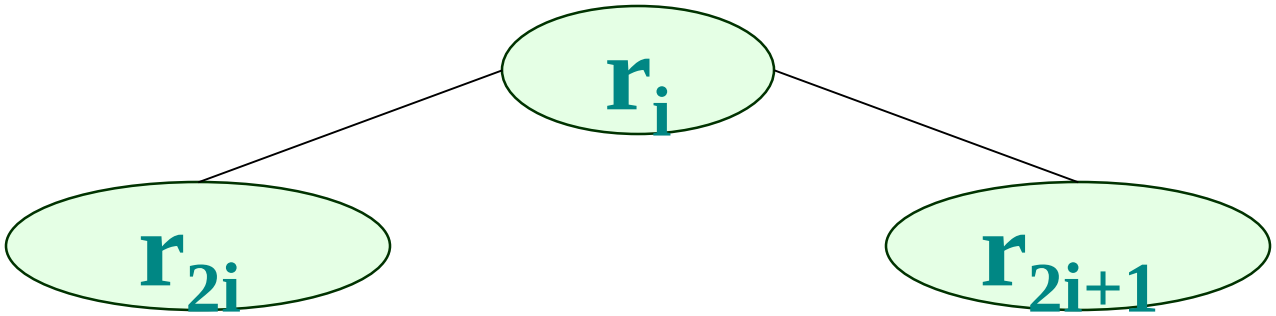
$$\begin{cases} r_i \le r_{2i} \\ r_i \le r_{2i+1} \end{cases} \text{Min-Heap} \quad \textbf{or} \quad \begin{cases} r_i \ge r_{2i} \\ r_i \ge r_{2i+1} \end{cases} \text{Max-Heap}$$

for instance :

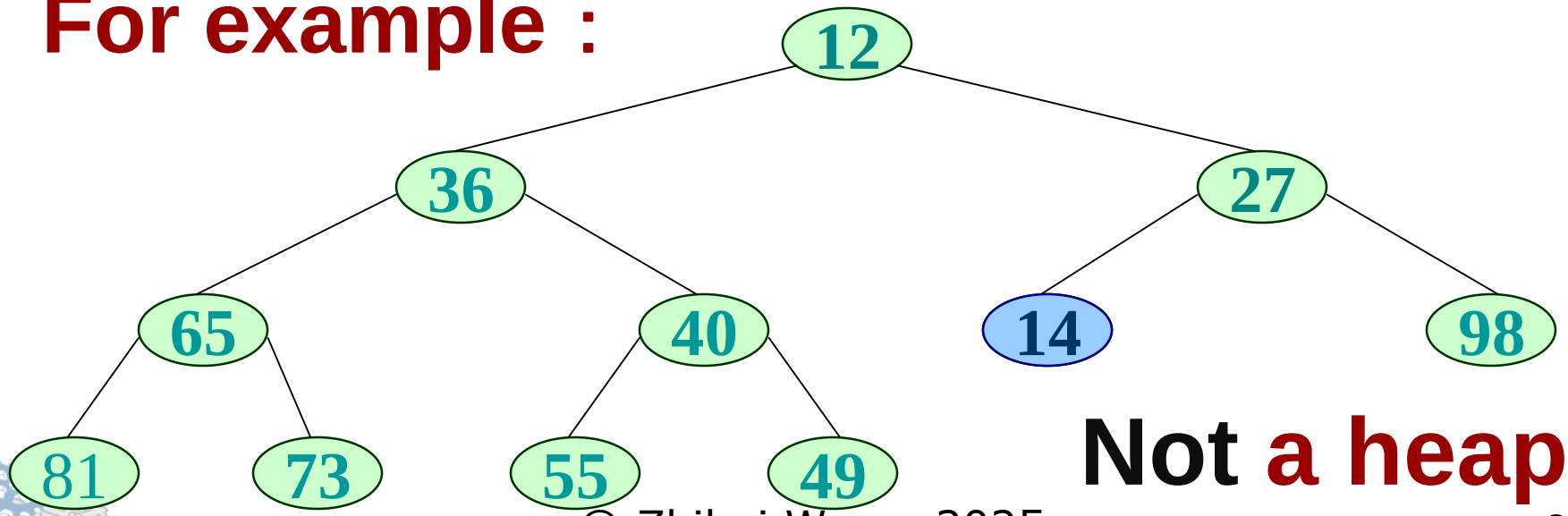
{12, 36, 27, 65, 40, 34, 98, 81, 73, 55, 49} It is a Min-Heap**.**

{12, 36, 27, 65, 40, 14, 98, 81, 73, 55, 49} **Not a heap**

If the sequence {12, 36, 27, 65, 40, 34, 98, 81, 73, 55, 49} is regarded as a complete binary tree, then $\mathbf{r_{2i}}$ is the left child of $\mathbf{r_i}$, and $\mathbf{r_{2i+1}}$ is the right child of $\mathbf{r_i}$.



For example :



Not a heap

Heap sort is a method of sorting a sequence of records by using the properties of heap.

For example :

{ 40, 55, 49, 73, 12, 27, 98, 81, 64, 36 }

Build a max-heap

{ 98, 81, 49, 73, 36, 27, 40, 55, 64, 12 }

Exchange 98 and 12

{ 12, 81, 49, 73, 36, 27, 40, 55, 64, 98 }

Sift re-adjust to a max-heap

{ 81, 73, 49, 64, 36, 27, 40, 55, 12, 98 }

Defining the Heap Type

typedef SqList HeapType;

// The heap is represented by a sequential list

Two problems need to be solved to implement heap sort:

① How to build a heap from an unordered sequence?

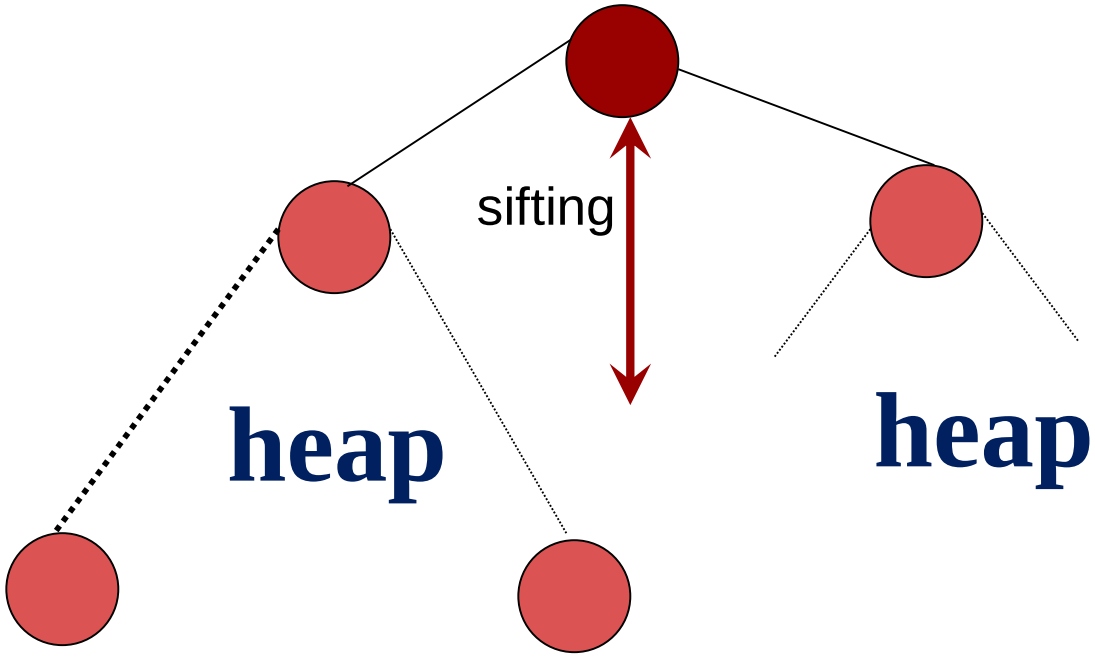
② How to adjust the remaining elements into a new heap after outputting the top element?

How to adjust the remaining elements into a new heap after outputting the top element?

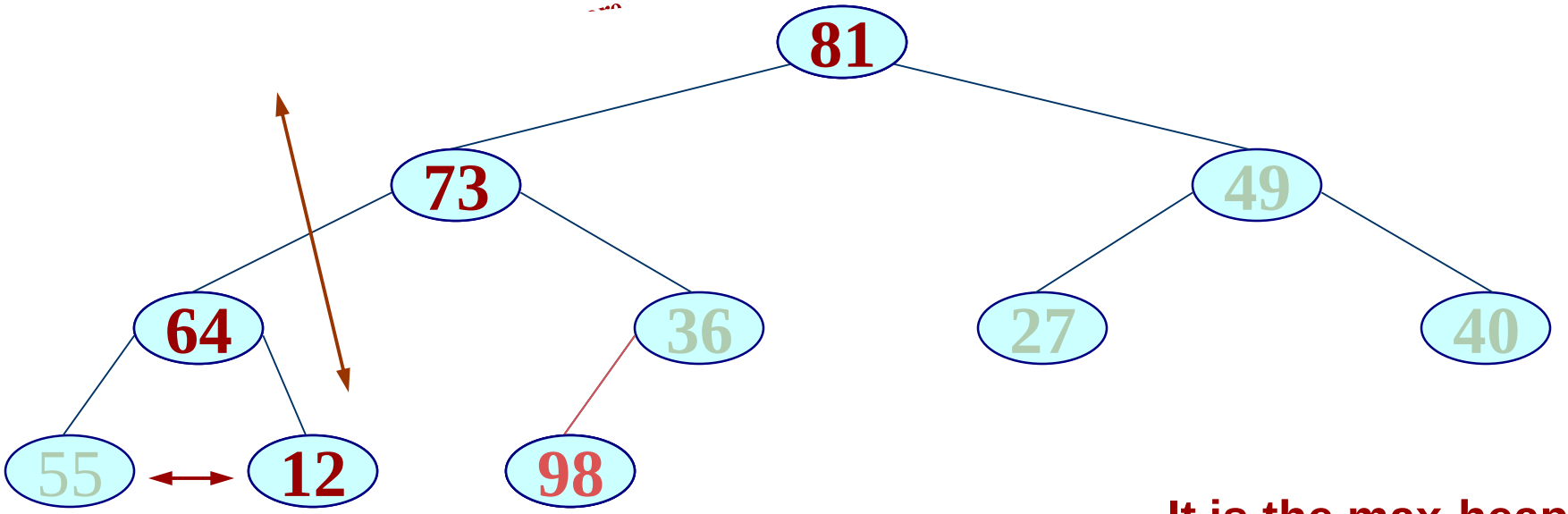
If the last element in the heap replaces the top element, and both left and right subtrees of the root node are heaps, only top-down adjustments are required. The adjustment process from the heap top to the leaf is called "sifting".

The so-called "sifting" refers to adjusting the root node of a complete binary tree (whose left and right subtrees are already heaps) so that the entire binary tree becomes a heap.

Characteristics of a complete binary tree for a heap: The sequence of nodes along the path from any node in the tree to the root is ordered by their keys.



for instance:



It is the max-heap.

But after the swap between 98 and 12, it was no longer a heap.

Therefore, it needs to be "filtered".

```
void HeapAdjust (RcdType &R[], int s, int m)
{ // Except for R[s], all keywords in the known R[s..m] are
// This function adjusts R[s] from top to bottom to satisfy heap characteristics
// The keyword makes R[s..m] also a  max-heap

    rc = R[s];              //  Temporary storage of R[s]

    for ( j=2*s; j<=m; j*=2 ) {  // j initial value points to the left child
    Top-down screening;
    }

    R[s] = rc; // Insert the heap top record before adjustment into position s

} // HeapAdjust
```

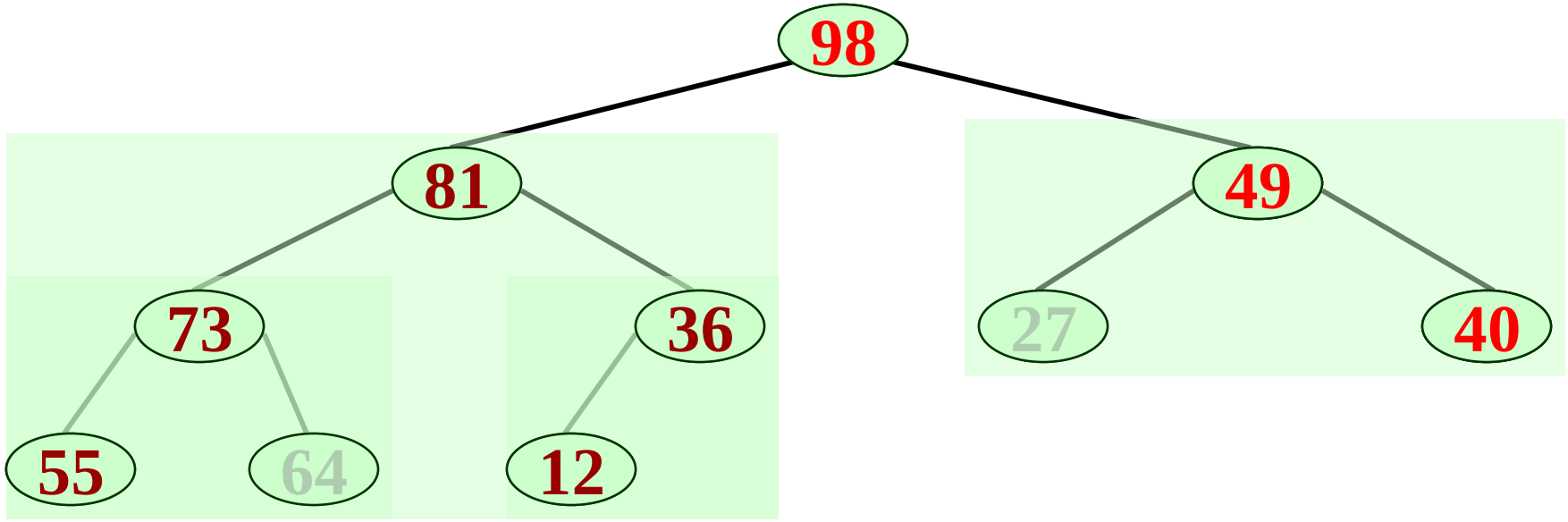
```
if ( j<m && R[j].key<R[j+1].key ) ++j;
// Compare left/right subtree roots first
// Let j indicate the position of the record with the larger key

if (rc.key >= R[j].key )  break;
// Re-compare the root and child tree root,
// If the ">=" condition is true, it indicates that the rc plug has been found
// Enter position s. Do not continue adjusting downward

R[s] = R[j];   s = j;
```

Otherwise, the record will move up and needs further adjustment.

```
void HeapAdjust (RcdType &R[], int s, int m)
{    // Max-heap
     rc = R[s];          // Temporarily store R[s]
     for ( j=2*s; j<=m; j*=2 ) {  // j initially points to the left child
        if ( j<m && R[j].key<R[j+1].key )  ++j;
        if ( rc.key >= R[j].key )  break;
        R[s] = R[j];   s = j;
     }
     R[s] = rc; // Insert the original top node into position
} /s HeapAdjust
```

Building a heap is a process of "sifting" from bottom to top.

Example: The keyword sequence before sorting (building a max-heap)



Now that both left and right subtrees have been adjusted to heap structures, the final step is to adjust the root node to make the entire binary tree a heap.

```
Typedef  SqList  HeapType; //The heap uses sequential list storage

Void HeapSort(HeapType  &H) {

 //Sort the sequential list H using heap sort

 for (i=H.length/2; i>0; --i)  //Build H.r[1..H.Length] into a max-heap

    HeapAdjust(H,i,H.length);

 for (i=H.length; i>1; --i) {

    H.r[1]←→H.r[i];      //Swap the heap top (H.r[1]) with the
                         //last element of the unsorted sub-sequence H.r[1..i]

    HeapAdjust(H,1,i-1);  //Re-adjust H.r[1..i-1] to a max-heap

  }

}
```

Time complexity analysis of heap sort:

1.Keywords required for "filtering" in a heap of depth k

The maximum number of comparisons is 2(k-1).

2.For n keywords, build a heap of depth $h(=\lfloor log_2 n \rfloor+1)$, and the number of key comparisons required is at most 4 n;

3.Adjust the "top heap" n-1 times, totaling the key

The number of character comparisons does not exceed

$$2\ (\lfloor log_2(n-1) \rfloor + \lfloor log_2(n-2) \rfloor + \ \ldots + log_2 2) < 2n(\lfloor log_2 n \rfloor)$$

Therefore, the time complexity of heap sort is $\mathbf{O(}n\mathbf{log}n\mathbf{)}$.

Chapter Contents

- **Insertion Sort**
- **Exchange Sort**
- **Selection Sort**
- **Merge Sort**
- Radix Sorting
- A Comprehensive Comparison of Various Sorting Methods
- External Sort

10.5 Merge Sort

The process of merge sort is based on the following basic ideas:

Merge two or more ordered sub-sequences into one ordered sequence.

In internal sorting, the 2-way merge sort is typically used. This means that two **adjacent records** are merged into a sorted subsequence.

Ordered subsequence R[i..*m*]	**Ordered subsequence R[m+1..*n*]**

Merge into an ordered sequence of records.

Ordered sequence R[i..*n*]

This operation is effortless for sequential lists.

```
void Merge (RcdType SR[], RcdType &TR[],
                                    int i, int m, int n) {
// The ordered record sequence SR[i..m] and SR[m+1..n]
// Merge into an ordered record sequence TR[i..n]

 for (j=m+1, k=i;  i<=m && j<=n;  ++k)
  {   // Merge SR records into TR in ascending order
    if (SR[i].key<=SR[j].key)  TR[k] = SR[i++];
    else TR[k] = SR[j++];
  }
    …   …
} // Merge
```

```
if (i<=m) TR[k..n] = SR[i..m];
// Copy the remaining SR[i..m] to TR

if (j<=n) TR[k..n] = SR[j..n];
// Copy the remaining SR[j..n] to TR
```

The idea of merge sort for 2-way merge:

If the initial sequence contains n records, it can be regarded as n ordered sub-sequences, each of length 1. Then merge them pairwise to obtain $\lceil n/2 \rceil$ ordered sub-sequences of length 2 or 1. Continue merging pairwise until an ordered sequence of length n is obtained.

The process of 2-way merge sort

The sequence to be sorted is (25, 57, 48, 37, 12, 92, 86)
Sequence state after each pass of 2-way merge sort:

Initial state	[25]	[57]	[48]	[37]	[12]	[92]	[86]
1st pass merge	[25 57]		[37 48]		[12 92]		[86]
2nd pass merge	[25 37 48 57]				[12 86 92]		
3rd pass merge	[12 25 37 48 57 86 92]						

```
void Msort ( RcdType SR[],
                    RcdType &TR1[], int s, int t ) {
            // Merge sort SR[s..t] into TR1[s..t]
  if (s= =t) TR1[s]=SR[s];
  else
   {          …   …
    }
} // Msort
```

m = (s+t)/2;

// Divide SR[s..t] into SR[s..m] and SR[m+1..t]

Msort (SR, TR2, s, m);

// recursively merge SR[s..m] into an ordered TR2[s..m]

Msort (SR, TR2, m+1, t);

// recursively merge SR[m+1..t] into an ordered TR2[m+1..t]

Merge (TR2, TR1, s, m, t);

// Merge TR2[s..m] and TR2[m+1..t] into TR1[s..t]

```
void MergeSort (SqList &L) {
        // Perform 2-way merge sort on the sorted list L
    MSort(L.r, L.r, 1, L.length);
} // MergeSort
```

Clearly, the time complexity of each merge operation is $O(n)$.

A total of $\lceil \log_2 n \rceil$ trips are required.

The time complexity of merge sort for n records is

$O(n\log n)$. The space complexity is $O(n)$.

Chapter Contents

- **Insertion Sort**
- **Exchange Sort**
- **Selection Sort**
- **Merge Sort**
- **Radix Sorting**
- A Comprehensive Comparison of Various Sorting Methods
- External Sort

10.6 Radix Sort

Radix sort is an internal sorting algorithm that implements "single-key sorting" by leveraging the idea of "multi-key sorting". Radix sort **does NOT require key comparisons**.

Previous sorting methods are implemented through **two operations: key comparison and record movement**.

- **sorting by multiple keywords**
- **chain radix sort**

1. Multi-Key Sorting

a sequence of n records **{ R_1, R_2, …, R_n}**

The ordering of keywords (K_i^0, K_i^1,…,K_i^{d-1})

is defined as:

For any two records R i and R j in the sequence

For all (1≤i<j≤n), the following ordered relation

(as defined in the dictionary) must be **satisfied**:

(K_i^0, K_i^1, …,K_i^{d-1}) < (K_j^0, K_j^1, …,K_j^{d-1})

Among them, K^0 is called the "most significant" key

K^{d-1} is called the "least significant" key

There are two common approaches to implement multi-key sorting:

- Most Significant Digit (MSD) method
- Least Significant Digit (LSD) method

Most Significant Digit (MSD) method

First, sort K^0 and divide the record sequence into several sub-sequences according to different values of K^0 (each sub-sequence contains records with the same K^0). Then, sort each sub-sequence separately according to K^1, and so on**.**..., and so on, until the last keyword is sorted.

Least Significant Digit (LSD) method

First, sort $\mathbf{K^{d-1}}$, then $\mathbf{K^{d-2}}$, and so on until the most dominant keyword $\mathbf{K^{0}}$ is sorted.

In the process of sorting, each keyword is the whole sequence to participate in the sorting.

Note: **The LSD method requires stable sorting for all keyword bits except the least significant bit.**

For example: The student record contains three keywords:

Department, **class number**, and **serial number within the class**, with department as the primary keyword.

The LSD sorting process is as follows:

disordered sequence	3,2,30	1,2,15	3,1,20	2,3,18	2,1,20
K2 sort	1,2,**15**	2,3,**18**	3,1,**20**	2,1,**20**	3,2,**30**
K1 sort	3,**1**,20	2,**1**,20	1,**2**,15	3,**2**,30	2,**3**,18
K0 sort	**1**,2,15	**2**,1,20	**2**,3,18	**3**,1,20	**3**,2,30

2. Linked Radix Sort

For numeric or character-type single keys, they can be treated as multi-keys composed of multiple digits or characters. In this case, the "distribution-collection" method can be used for sorting, which is called the radix sort algorithm.

For example: the following set of keywords

{209, 386, 768, 185, 247, 606, 230, 834, 539 }

First, **distribute** them into 10 groups based on the value of their **units digit** (0, 1, ..., 9), then **collect** them in order from 0 to **9:**

{230,834,185,386,606,247,768,209,539}

Next, **distribute** them into 10 groups based on the value of their **tens digit** (0, 1, ..., 9), then **collect** them again in order from 0 to 9:

{606,209,230,834,539,247,768,185,386}

Repeat the above operation for the **hundred digit**.

{185,209,230,247,386,539,606,768,834}

When implementing radix sort on a computer, linked lists should be used as the storage structure (i.e., linked radix sort) to reduce the required auxiliary storage space. The specific steps are as follows:

1. The records to be sorted are linked by pointers to form a linked list;

2. During the "allocation" process, records are assigned to different "linked queues" based on their current "key bit" values, with all records in each queue sharing the same "key bit".

3. When "collecting", the values of the current keyword bit are taken from small to large, and the first and last of each queue are linked together to form a linked list;

4. Repeat steps 2) and 3) for each keyword position.

Example:

p→369→367→167→239→237→138→230→139

First Distribution (by units digit)

0 Head f[**0**]→23**0**←r[**0**] 0 Tail

7 Head f[7]→36**7**→16**7**→23**7**←r[7] 7 Tail

8 Head f[**8**]→13**8**←r[**8**] 8 Tail

9 Head f[**9**]→36**9**→23**9**→13**9**←r[**9**] 9 Tail

First Collection (concatenate queues in order)

p→230 →367→167→237 →138 →369→239→139

p→230→367→167→237→138→369→239→139

Second Distribution (by tens digit)

f[**3**]→2**3**0→2**3**7→1**3**8→2**3**9→1**3**9←r[**3**]

f[**6**]→3**6**7→1**6**7→3**6**9←r[**6**]

Second Collection

p→230→237→138→239→139 →367→167→369

p→230→237→138→239→139→367→167→369

Third Distribution (by hundreds digit)

f[1]→138→139→167←r[1]

f[2]→230→237→239←r[2]

f[3]→367→369←r[3]

After Third Collection, the sorted sequence is obtained:

p→138→139→167 →230→237→239 →367→369

The time complexity of radix sort is O(d(n+rd)).

Among them: allocation is O(n)

Collects in O(rd) (rd is "base")

d is the number of trips for "allocation-collection"

Chapter Contents

- **Insertion Sort**
- **Exchange Sort**
- **Selection Sort**
- **Merge Sort**
- **Radix Sorting**
- **A Comprehensive Comparison of Various Sorting Methods**
- External Sort

10.7

A Comprehensive Comparison of Various Sorting Methods

I. Time Performance

Insertion Sort, Bubble Sort, Quick Sort, Simple Selection, Heap Sort, Merge Sort, Radix Sort

1.average time performance

Time complexity is O(*n*log*n*):

Quick sort, heap sort, and merge sort

Time complexity is O(n^2):

Insertion sort, bubble sort, and

Simple Selection Sort

Time complexity is O(n):

radix sorting

2. When the sequence of records to be sorted is already ordered by key:

Straight insertion sort and bubble sort achieve a time complexity of **$O(n^2)$**.The time performance of quicksort degrades to **$O(n)$**.

3.The time performance of simple selection sort, heap sort, and merge sort does **not change with the distribution** of keys in the record sequence.

2. Space Performance

It refers to the auxiliary space required for sorting.

1.The space complexity of all **simple sorting methods** (including direct insertion, bubble sort, and simple selection) and **heap sort** is **O(1)**.

2.Quick sort is O(log*n*), the auxiliary space required by the stack during the execution of the recursive program**;**

3.Merge sort requires the most auxiliary space, and its space complexity is **O(*n*).**

4.The space complexity of the **chain radix sort is O(*rd*)** because of the queue pointer.

3.Stability of Sorting Methods

1. A stable sorting method means that for two records with equal keys, their relative positions in the sequence remain unchanged before and after sorting.

Before sorting: $\{ \cdot\cdot\cdot\cdot\cdot R_i(K) \cdot\cdot\cdot\cdot\cdot R_j(K) \cdot\cdot\cdot\cdot\cdot \}$

After sorting: $\{ \cdot\cdot\cdot\cdot\cdot R_i(K)\ R_j(K) \cdot\cdot\cdot\cdot\cdot\cdot\cdot\cdot\cdot\cdot \}$

2. When sorting a multi-key record sequence using the LSD (Least Significant Digit first) method, all key positions (except the least significant digit) must use stable sorting methods.

for instance：

Sort before (56,34,**47**,23,66,18,82,**47**)

If sorting produces the result
(18, 23, 34, **47** , **47** , 56, 66, 82)
then the sorting method is **stable**.

If sorting produces the result
(18, 23, 34, **47** , **47** , 56, 66, 82)
The sorting method is called **unstable.**

3.For unstable sorting methods, just provide one example to illustrate.

For example, sort the array {4,3,4,2} quickly.

Get {2,3,4,**4**}

4.**Quick sort,** direct selection sort**, heap sort,** and **Shell sort** are **an unstable sorting method**.

4. On the Lower Bound of Time Complexity of Sorting Method

Except base sort, all the other methods discussed in this chapter are **based on the comparison of keywords.**

It can be shown that the **fastest time complexity achievable by such sorting methods** is O(n log n). (Radix sort, which does not rely on comparing key values, is exempt from this constraint.)

Chapter Contents

- Insertion Sort
- Exchange Sort
- Selection Sort
- Merge Sort
- Radix Sorting
- A Comprehensive Comparison of Various Sorting Methods
- External Sort

1 .Problem Statement

1. When the number of records to be sorted is extremely large (too large to fit into memory all at once), the sorting methods discussed in previous sections cannot be applied.

2. Basic Process of External Sorting

It consists of two relatively independent steps:

1. According to the size of available memory, several ordered sub-sequences of records are constructed **by using the internal sorting method, which are usually called "merging segments" in external memory.**

2. Through the "merger", the length of the ordered subsequence of the record is gradually expanded until the whole record sequence **in the external storage is ordered by the key word.**

For example, consider a disk containing 10,000 records, documents, and the current computer can only handle
If it can sort 1000 records internally, then
Firstly, the internal sorting method is used to obtain 10
Merge the initial segment first, then merge step by step.

Assume that 2-merge (i.e. merge two by two), then

The first merge was from 10 merge segments to 5 merge segments.

The second merge resulted in 3 merge segments from 5 merge segments.

The third merge is to get two merge segments from three merge segments.

The last merge gets the ordered sequence of the whole record.

Analyzing the number of external storage accesses (read/write operations) during the external sorting process:

Assume the "data block" size is 200, meaning each external memory access can read/write 200 records.

For 10,000 records, processing one round requires 100 external memory accesses (50 reads and 50 writes).

Thus, for the above example**,**

- 1) Accessing external storage 100 times is required to obtain 10 initial merge segments;
- 2) Each merge operation requires 100 external memory accesses;
- 3) The total number of access to external memory is $100 + 4 \times 100 = 500$ times.

The total time of external sorting should also include the time required for internal sorting and the time for internal merging during multi-pass merging. Obviously, apart from the factor of internal sorting, **the time of external sorting depends on the number of passes needed for multi-pass merging.**

Example: If 5-way merging is used in the above case, only 2 merging passes are required. The total number of external storage accesses will be reduced to $100 + 2 \times 100 = 300$.

Except for the internal sort factor, **the external sort time depends on the number of passes required for each merge.**

In general, assuming that the sequence of pending records contains *m initial merge segments*, and *k-way merge* is used in the outer merge, the number of merge passes is $\lceil \log_k m \rceil$.

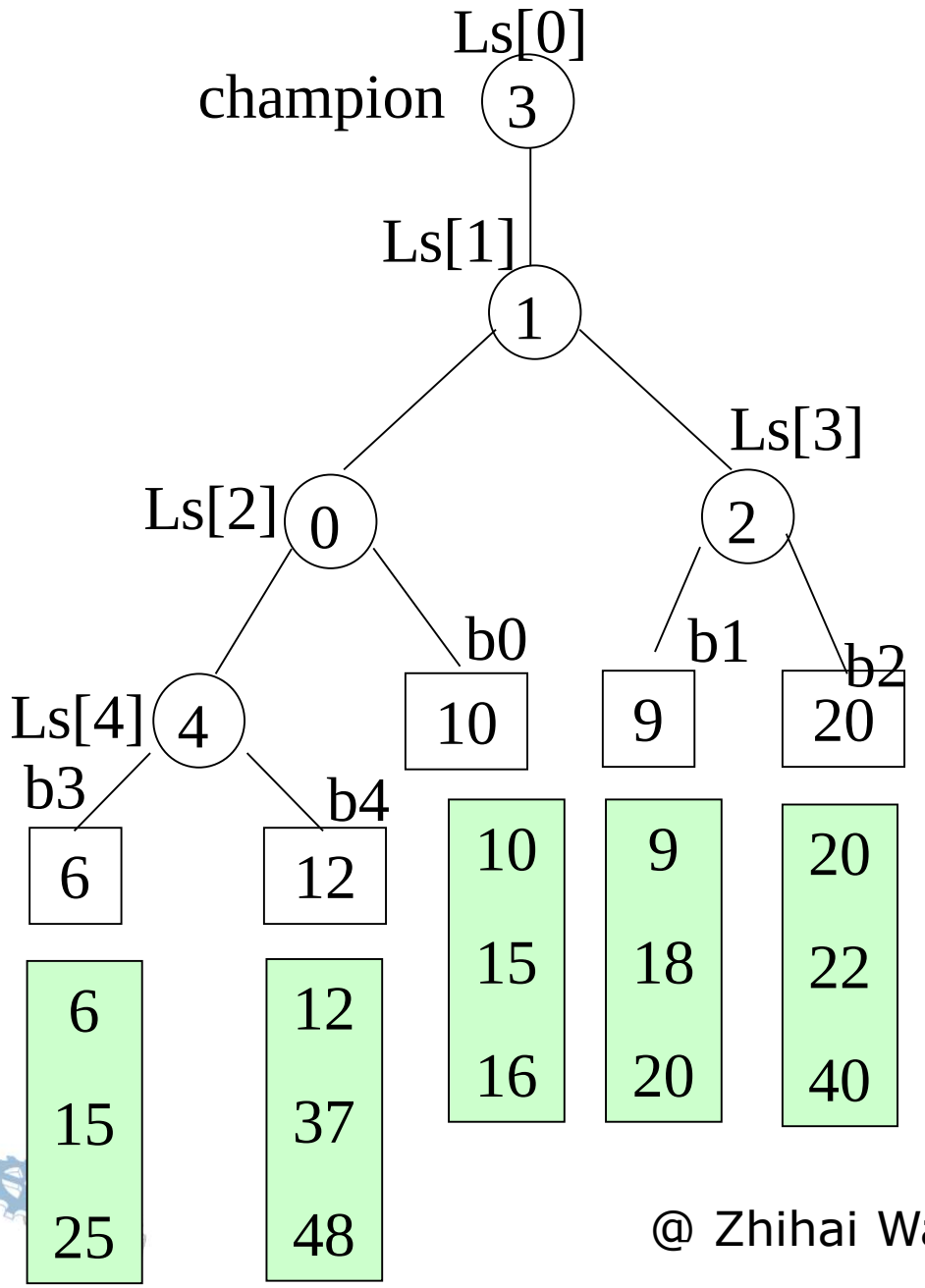
Obviously, the number of merge passes will decrease as k increases, and the number of external memory read/write operations will also decrease. Therefore, for external sorting, multi-pass merge is typically adopted. Additionally, *reducing m can also shorten the sorting time*.

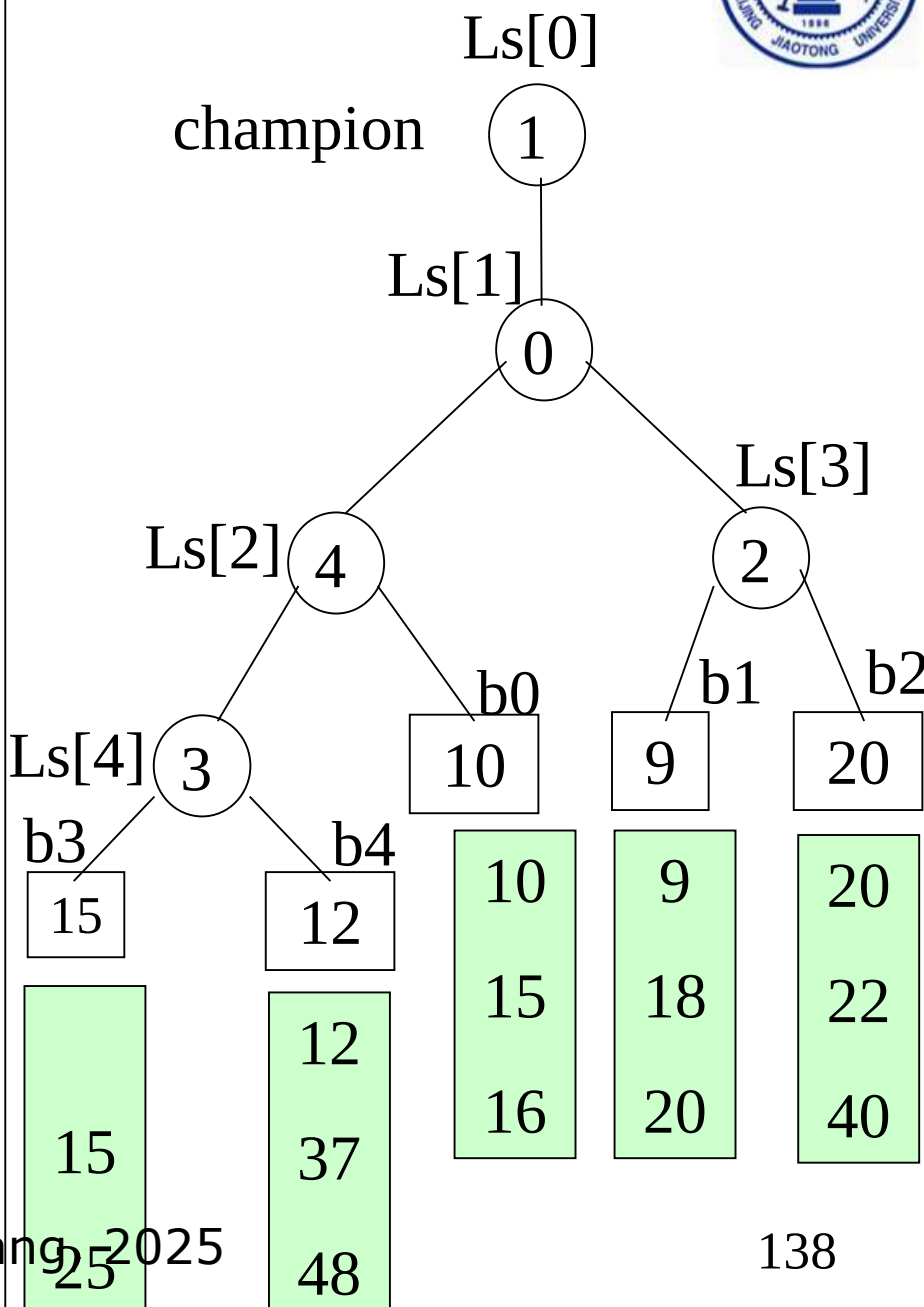
Common methods of external sort: loser tree, replacement selection sort, optimal merge tree

In general, assuming the sequence of records to be sorted contains m initial merge segments, and k-way merging is used for external sorting, the number of merge passes is $\lceil \log_k m \rceil$. Clearly, as k increases, the number of merge passes decreases, and the number of external read/write operations also decreases. Therefore, for external sorting, multi-way merging is typically adopted. The value of k can be chosen, but various factors must be considered comprehensively.

As k increases, the number of merges can be reduced, but the internal merge time will also increase, which will offset the benefit of reducing external memory read/write operations with increasing k. Using a loser tree, only $\lfloor \log_2 k \rfloor$ comparisons are required to select the smallest key among k records, ensuring that the merge time does not increase with k.

Loser Tree


Ls[0]
champion
3
Ls[1]
1
Ls[2]
0
Ls[3]
2
Ls[4]
4
b0
b1
b2
b3
b4
10
9
20
6
12
10
15
16
9
18
20
20
22
40
6
15
25
12
37
48


Ls[0]
champion
1
Ls[1]
0
Ls[2]
4
Ls[3]
2
Ls[4]
3
b0
b1
b2
b3
b4
10
9
20
15
12
10
15
16
9
18
20
20
22
40
15
25
12
37
48

From the formula

$\lceil \log_k m \rceil$ It can be seen that the sorting time is not only inversely proportional to k but also directly proportional to m. Therefore, reducing m can also reduce the sorting time.

K is the number of merge paths

m is the initial number of merge segments.

The sort-swap algorithm is derived from the tree sort algorithm. Its characteristics include: throughout the sorting procedure, the selection of the minimum (maximum) key and input/output operations can be performed either crosswise or parallelly.

Replacement-Selection Sort

Set:

The input file FI is the file to be sorted

The output file FO is the initial merge segment file

The working area WA in memory can hold W records.

Both WA and FO are initially empty.

Steps:

（1) fromFIimportWRecord to workspaceWA

（2) fromWASelect the record with the smallest keyword value.

Record, denoted asMINIMAXtake notes

（3） supportMINIMAXRecord output toFOMiddle Way

（4） likeFIIf not empty, followFIEnter the next record arriveWAcentre

（5) fromWAselection ratioMINIMAXlargest minimum Keyword records as newMINIMAXprice 。

（6） repeat(3)—(5)until inWANot selected Create a new record to get an initial merge segment. Generate a merge section end marker toFO

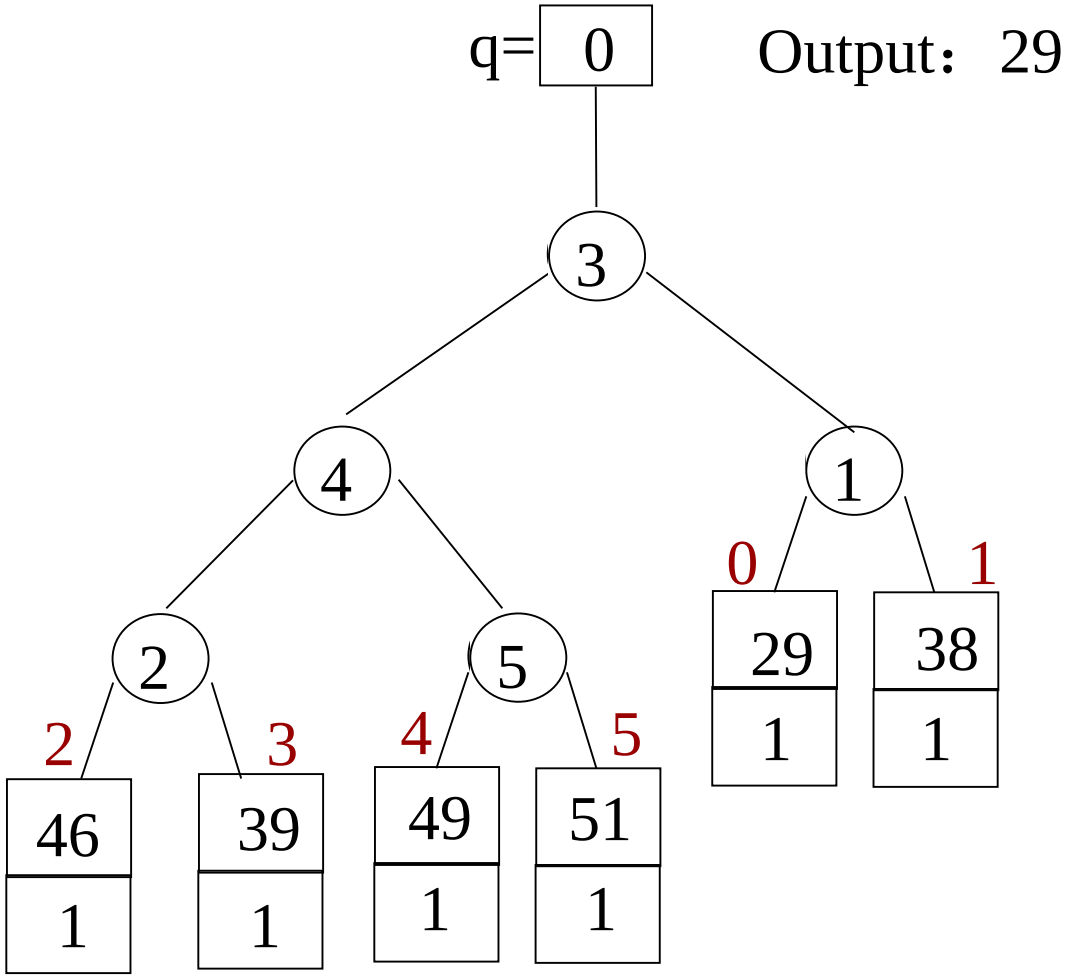
（7） repeat(2)—(6)， untilWAEmpty, get All initial merge segments.

Thus, the lengths of the initial merge segments are unequal.

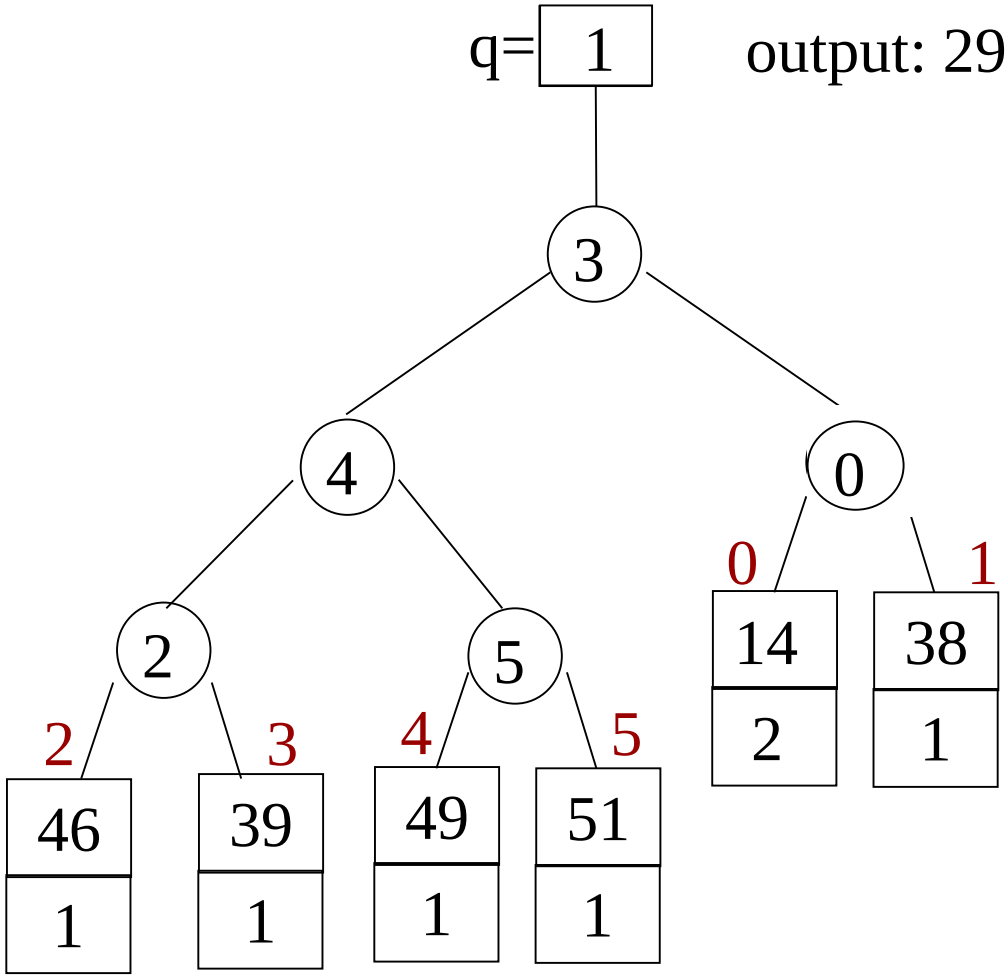
Replacement-Selection Sort

FO	WA (6)	FI
		51,49,39,46,38,29,14,61,15,30,1,48,52,3,63,27,4,…
	51,49,39,46,38,29	14,61,15,30,1,48,52,3,63,27,4,…
29	51,49,39,46,38	14,61,15,30,1,48,52,3,63,27,4,…
29	51,49,39,46,38,14	61,15,30,1,48,52,3,63,27,4,…
29,38	51,49,39,46, ,14	61,15,30,1,48,52,3,63,27,4,…
29,38	51,49,39,46,61,14	15,30,1,48,52,3,63,27,4,…
29,38,39	51,49, ,46,61,14	15,30,1,48,52,3,63,27,4,…
29,38,39	51,49,15,46,61,14	30,1,48,52,3,63,27,4,…
29,38,39,46	51,49,15, ,61,14	30,1,48,52,3,63,27,4,…
29,38,39,46	51,49,15,30,61,14	1,48,52,3,63,27,4,…
29,38,39,46,49	51, ,15,30,61,14	1,48,52,3,63,27,4,…
29,38,39,46,49	51,1,15,30,61,14	48,52,3,63,27,4,…
29,38,39,46,49,51	1,15,30,61,14	48,52,3,63,27,4,…
29,38,39,46,49,51	48,1,15,30,61,14	52,3,63,27,4,…
29,38,39,46,49,51,61	48,1,15,30, ,14	52,3,63,27,4,…
29,38,39,46,49,51,61	48,1,15,30,52,14	3,63,27,4,…
29,38,39,46,49,51,61,*	48,1,15,30,52,14	3,63,27,4,…
29,38,39,46,49,51,61,*,1	48, ,15,30,52,14	3,63,27,4,…
29,38,39,46,49,51,61,*,1	48,3,15,30,52,14	63,27,4,…

Replacement-Selection Sort Using a Loser Tree

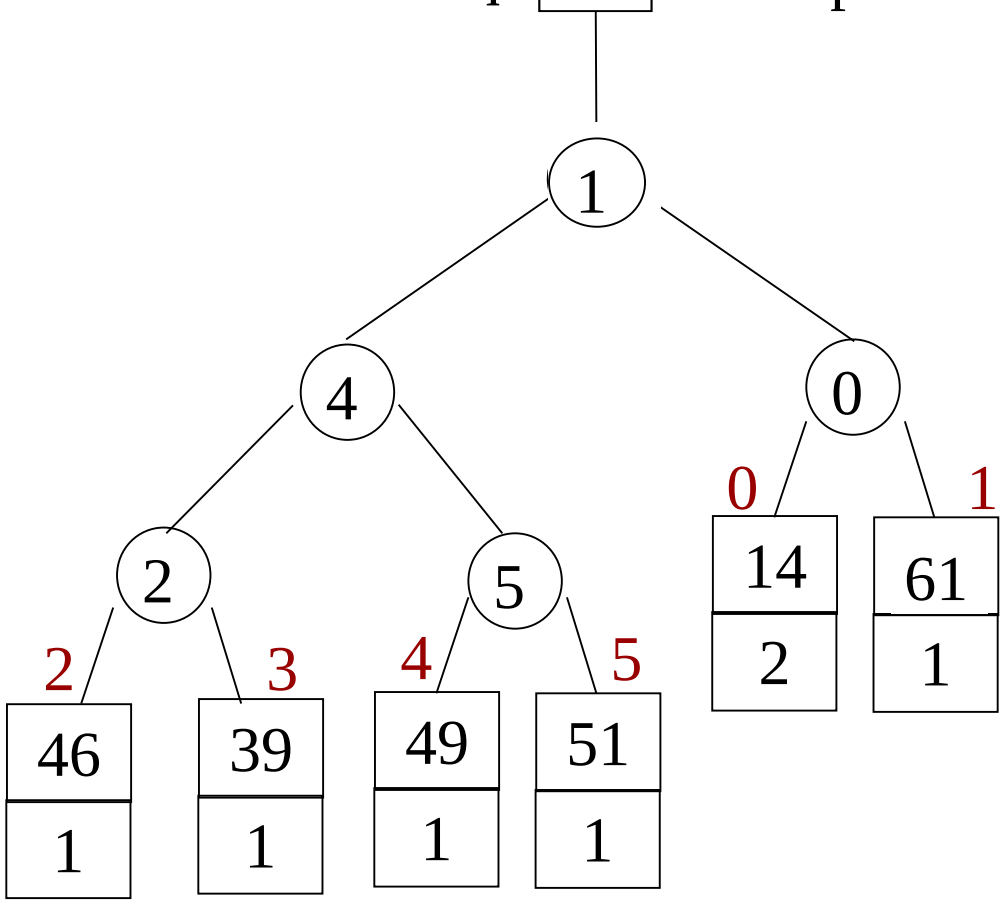


The "1" below each record represents the merge segment number.

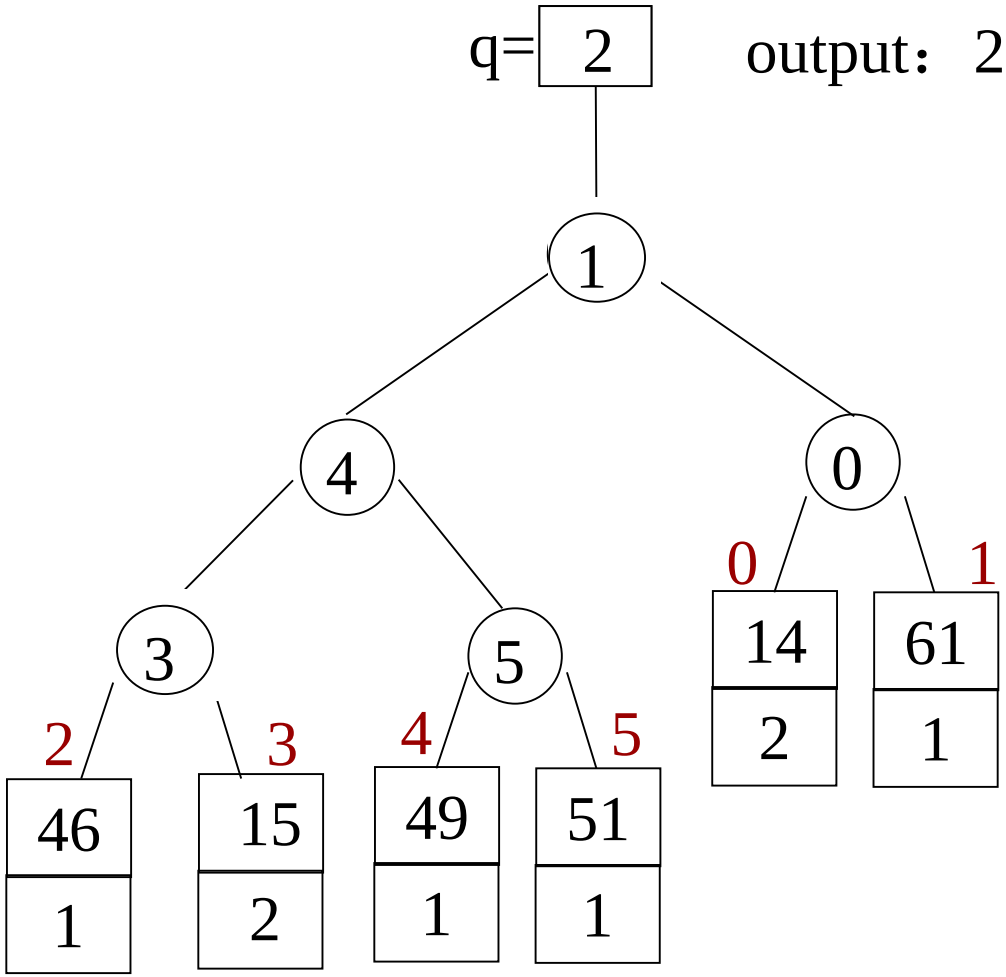


The 2 below represents the merge section number

q= 3 output：29，38，39

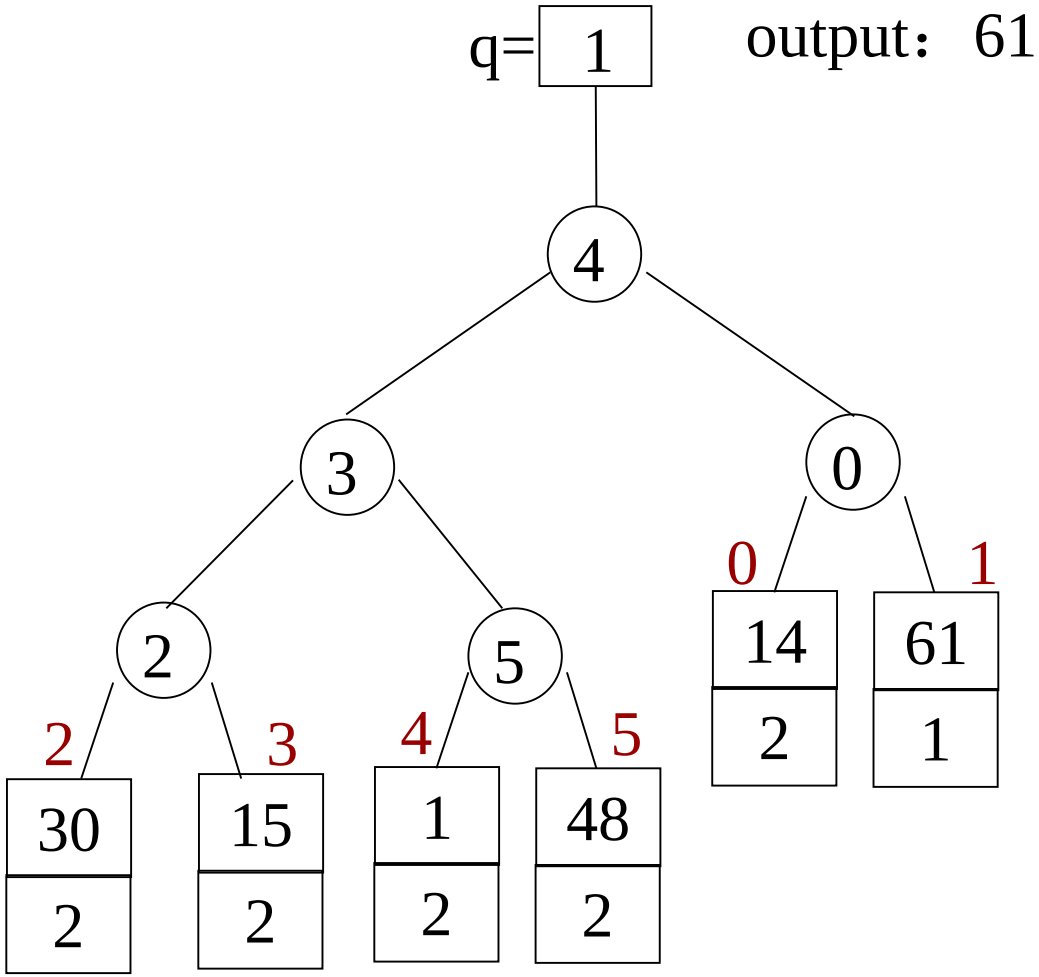


2

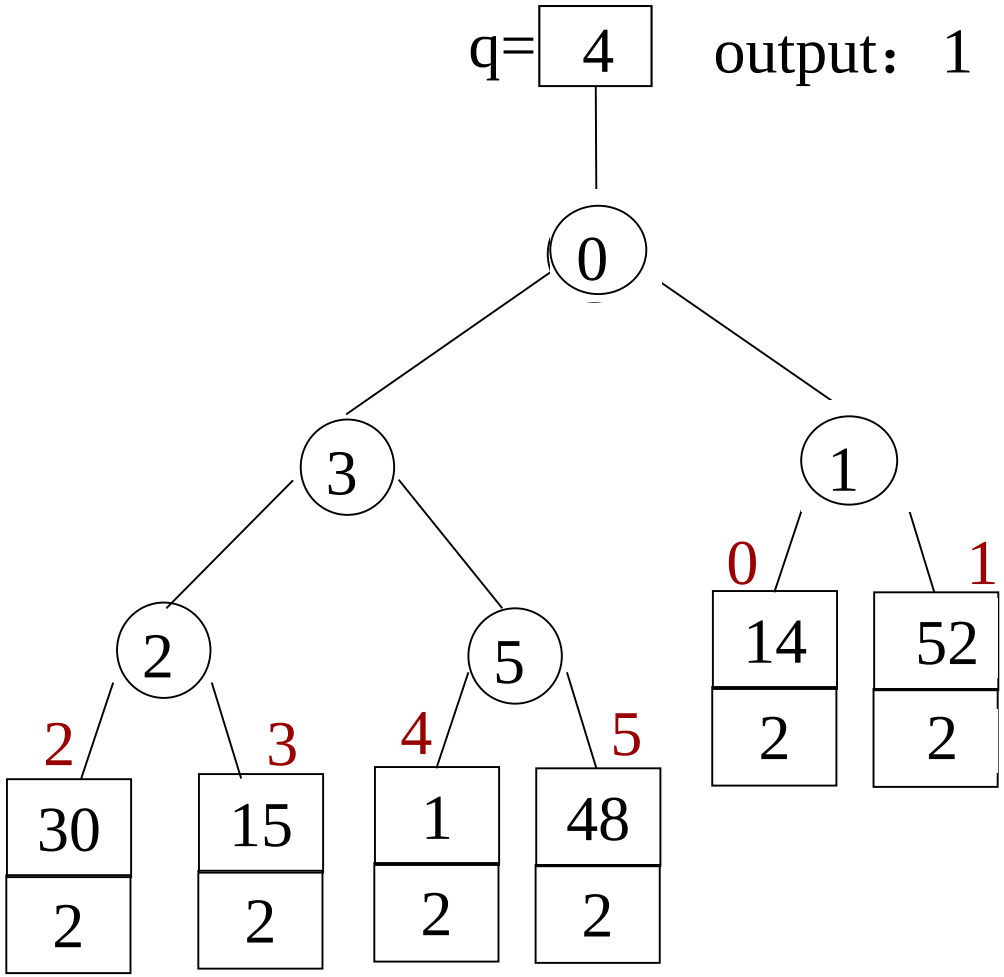


After outputting 49 and 51 in sequence:

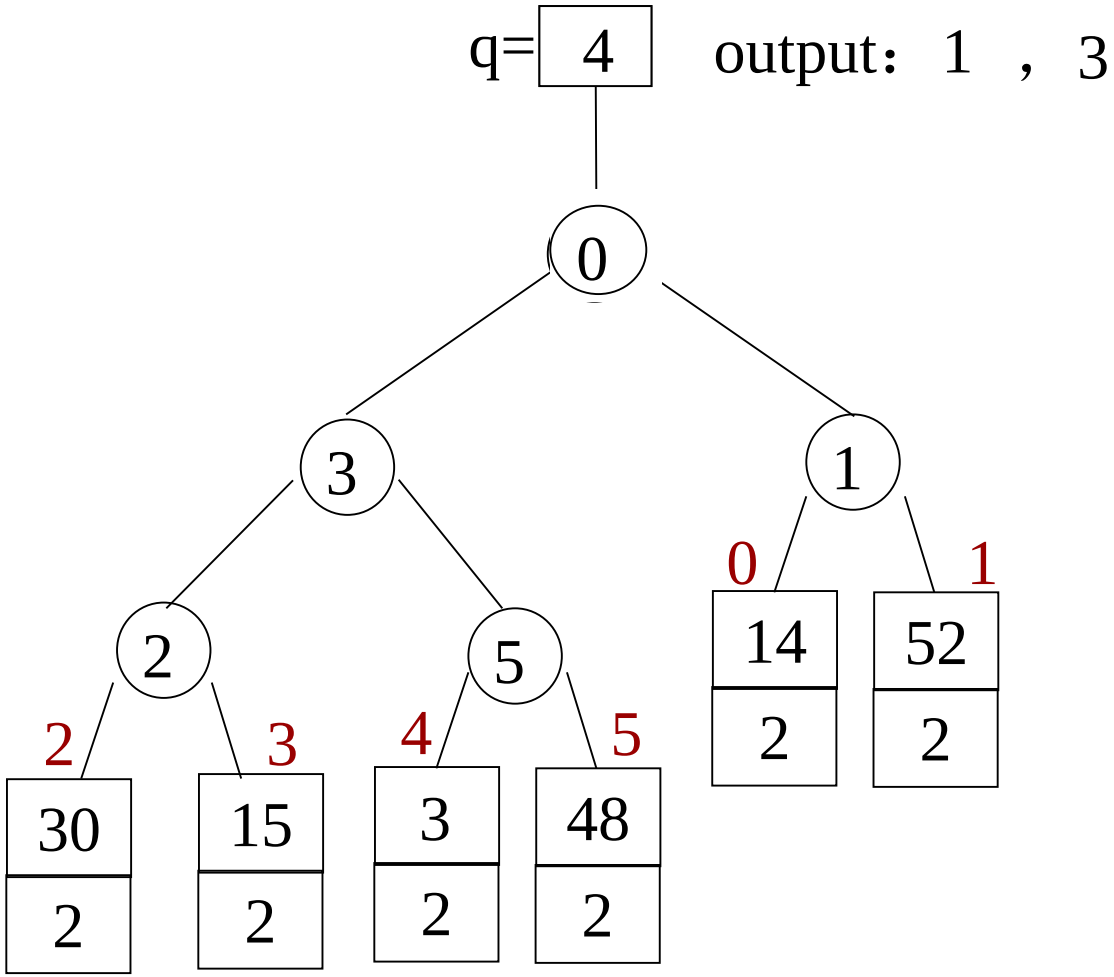
After outputting 49 and 51 in sequence:



After outputting: 29, 38, 39, 49, 51, 61



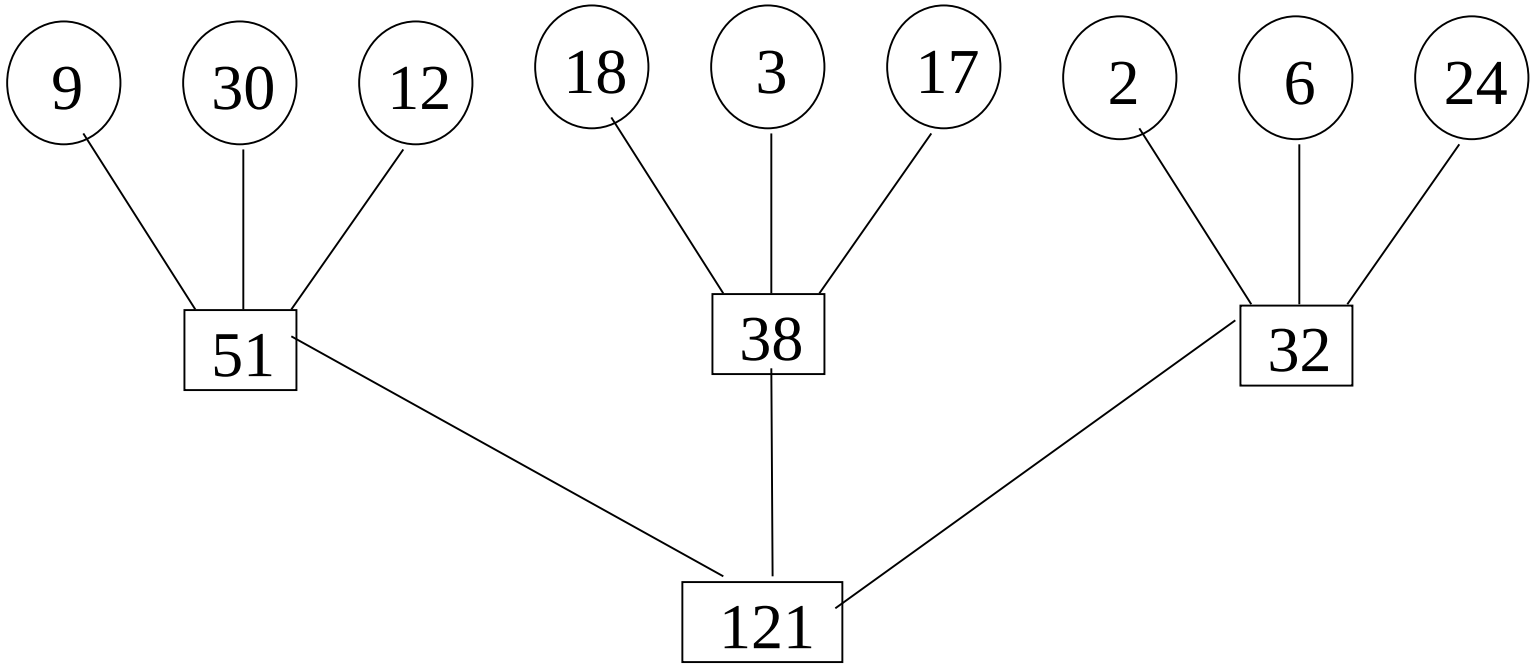
After outputting: 29, 38, 39, 49, 51, 61



Optimal Merge Tree

The lengths of the initial merge segments obtained by replacement-selection are unequal.

Segment lengths:9, 30, 12, 18, 3, 17, 2, 6, 24



Number of read/write operations:（9+30+12+18+3+17+2+6+24）*2*2=484

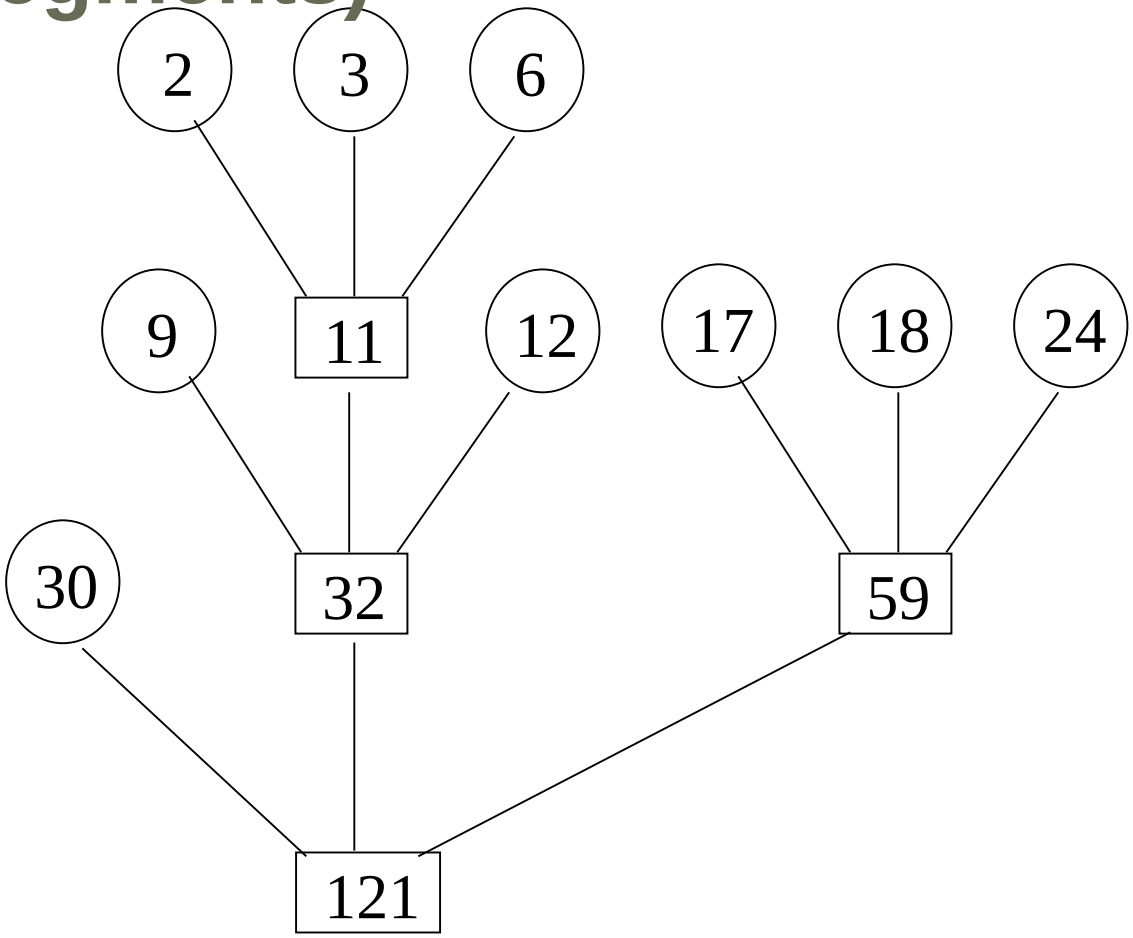
The nine segments mentioned above are merged into three groups, requiring two merging operations, resulting in a total read/write count of 484.

If the length of the initial merge segment is regarded as the weight of the leaf nodes in the merge tree, then the double of the weighted path length of this binary tree is 484.

If the merging scheme is different, the merging tree is different, and the weighted path length of the tree is different.

If we construct a Huffman tree as the merge tree for m initial merge segments of different lengths, the number of read and write operations of external memory can be minimized.

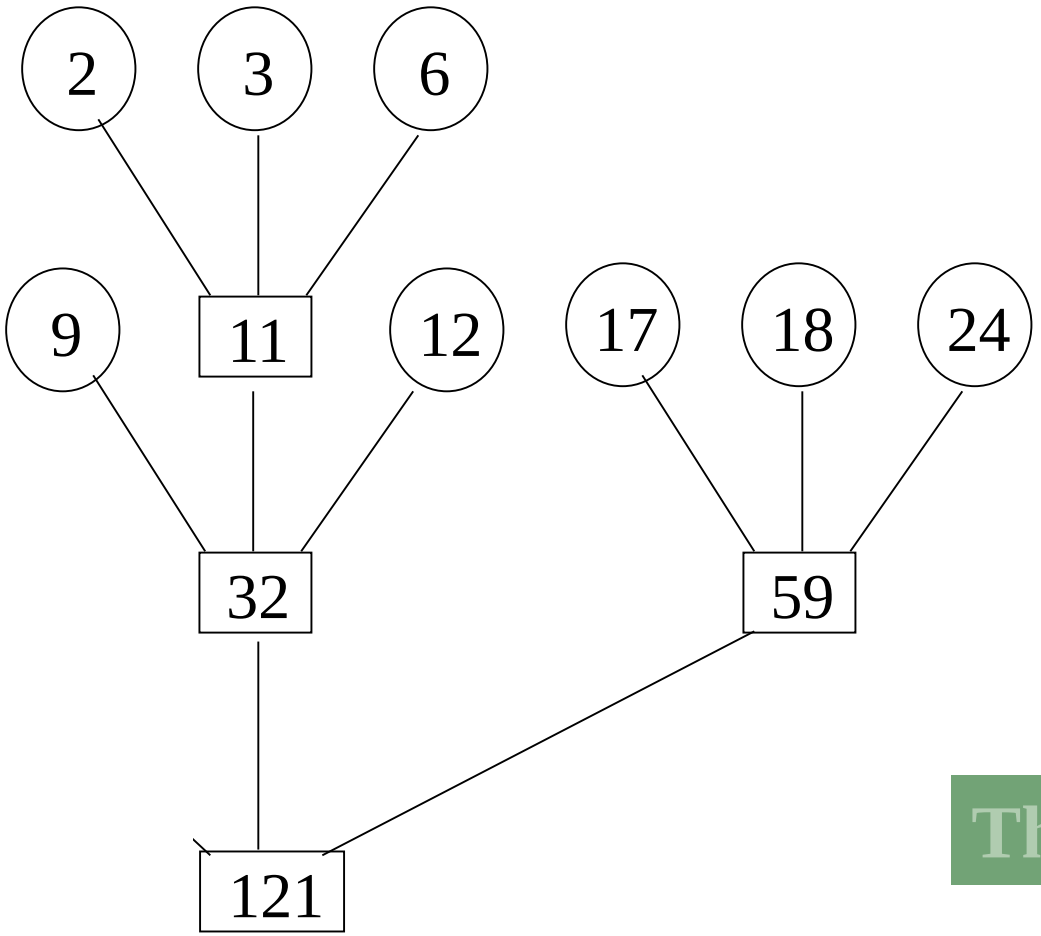
Optimal Merge Tree for 3-Way Merging (9 Segments)



Number of read/write operations:

[(3+2+6）*3+(9+12+17+18+24)*2+30*1]*2 =446

Merge Tree for 3-Way Merging (8 Segments)

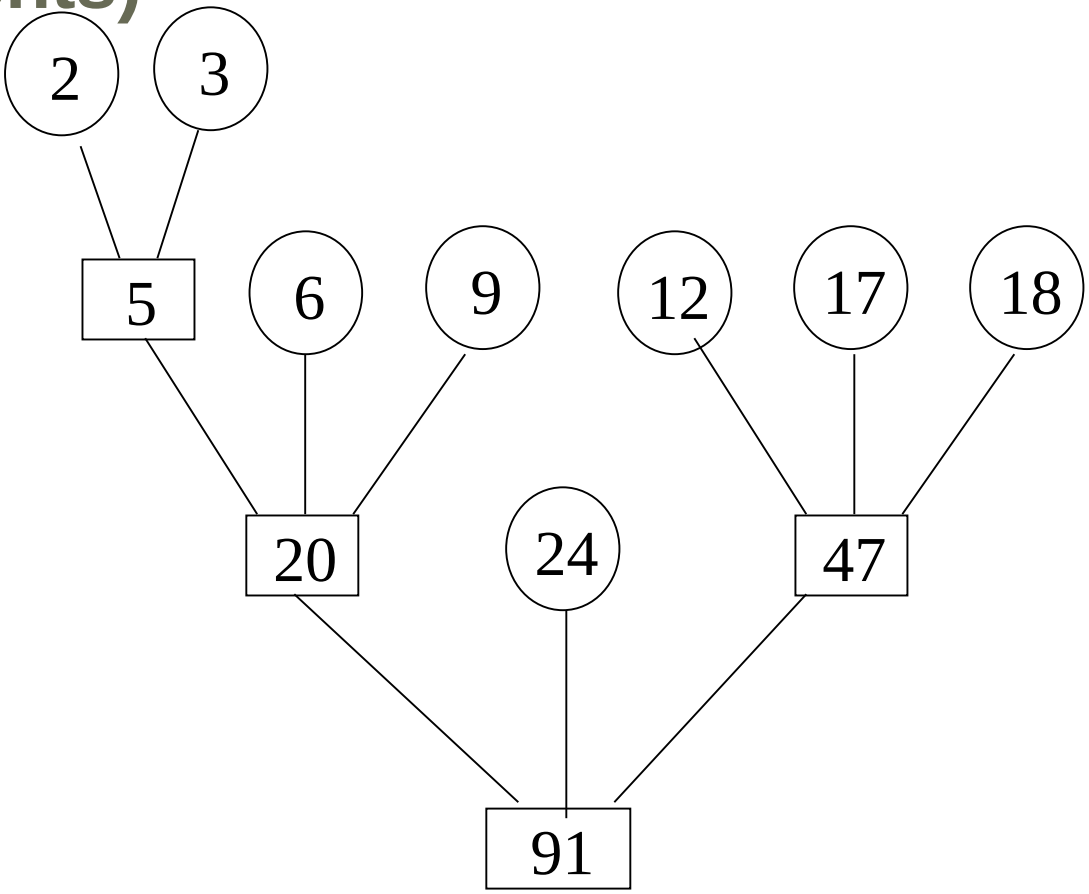


This is not optimal!

Number of read/write operations:

[(3+2+6）*3+(9+12+17+18+24)*2]*2 =386

Optimal Merge Tree for 3-Way Merging (8 Segments)



Solution: When the number of initial segments is insufficient, add a "virtual segment" with length 0.

Number of read/write operations:

$$[(3+2)*3+(6+9+12+17+18)*2+24]*2 = 326$$

How many dummy segments should be added?

For a three-way tree, the condition n3=(n0-1)/2 holds only when the degree of the node is 3 or 0.

Since n3 must be an integer, then

(n0-1) MOD 2 =0。

That is, for the 3-way merge, only when the initial merge segment number is even, a dummy segment is added.

chapter summary

1.Understand the definition of sorting and the characteristics of various sorting methods. Be familiar with the **sorting process of various methods and the principles on which they are based**. The sorting methods can be divided into five categories according to the different principles on which the sorting process is based, such as **insertion sort, exchange sort, selection sort, merge sort and radix sort**.

2.Master the analysis method for the **time complexity** of various sorting methods. Be able to analyze the **average-case** and **worst-case** time performance of sorting algorithms based on the "**number of key comparisons**".

According to the average time complexity, the internal sorting can be divided into three categories: simple sorting with **$O(n^2)$**, efficient sorting with **O(nlogn)**, and radix sort with **O(dn)**.

3. Understand the meaning of "stable" or "unstable" in sorting methods, and identify the scenarios where a stable sorting method is required.

4. The basic process of external sorting and its time analysis are introduced.

Homework: 10.1, 10.3, 10.6, 10.12,10.33

10.1 Using the key code sequence (503,087,512,061,908,170,897,275,653,426) as an example, manually execute the following sorting algorithm and record the key code state after each sorting pass:

(1) Insertion Sort (2) Shell Sort (increment d[1]=5)

(3) Quick Sort (4) Heap Sort

(5) Merge sort (6) Radix Sort

10.3 Among the sorting methods listed in question 10.1, which are stable and which are unstable? For each unstable method, provide an example of its instability.

10.6 The odd-even swap sorting algorithm operates as follows: First, for all odd indices i, compare a[i] with a[i+1]. Second, for all even indices i, compare a[i] with a[i+1]. If a[i]> a[i+1], swap the two. Then, repeat the process for odd indices, followed by even indices, and so on until the entire sequence is sorted.

(1) What is the termination condition of this sorting method?

(2) Analyze the number of key comparison times in the process of odd-even exchange sort when the initial sequence is in order or reverse order.

10.12 Determine whether the following sequence is a heap (min-heap or max-heap). If not, adjust it to a heap (with minimal record exchanges)

（1）（100、86、48、73、35、39、42、57、66、21）

（2）（12、70、33、65、24、56、48、92、86、33）

（3）（103、97、56、38、66、23、42、12、30、52、06、20）

（4）（05、56、20、23、40、38、29、61、35、76、28、100）

10.33 Try to implement the simple selection sort algorithm with single linked list as storage structure.

Thank You !